

---

# QUANTUM EEG SIGNAL CLASSIFIER

A Hybrid Classical–Quantum Machine Learning System  
for EEG Seizure Detection

---

**Document Type:** Complete Technical Project Report

**Model Version:** v1.3.0 (production)

**Technology Stack:** Python · Qiskit · MNE · scikit-learn · FastAPI · React · TypeScript

**Dataset:** CHB-MIT Scalp EEG Database (Subjects chb01–chb05)

**Report Generated:** 2026-09-03

**Classification System:** Variational Quantum Classifier (VQC)

**Artifact Location:** models/v1\_3\_0/

**Academic Context:** [No institution or supervisor data found in repository]

■ NOT FOR CLINICAL USE ■ RESEARCH PROTOTYPE ONLY

# Table of Contents

---

- 1. Executive Summary
- 2. Problem Statement
  - 2.1 EEG Signal Analysis
  - 2.2 Seizure Detection Challenges
  - 2.3 Motivation for Quantum ML
- 3. Project Objectives
- 4. System Requirements
- 5. Technology Stack
- 6. Complete System Architecture
  - 6.1 Training Architecture
  - 6.2 Inference Architecture
  - 6.3 Training vs Inference Separation
- 7. EEG Dataset — CHB-MIT
- 8. EEG Data Loading
- 9. EEG Validation
  - 10. Channel Selection
  - 11. Signal Filtering
  - 12. Segmentation
  - 13. Seizure Annotation and Labeling
  - 14. Feature Extraction
    - 14.1 Feature 1: Wavelet Approximation Energy
    - 14.2 Feature 2: Wavelet Detail Energy
    - 14.3 Feature 3: Shannon Entropy
    - 14.4 Feature 4: Delta Band Power
    - 14.5 Feature 5: Hjorth Mobility
    - 14.6 Feature Vector Construction
    - 14.7 Feature Implementation Audit Note
  - 15. Dataset Construction
  - 16. Subject-Aware Data Splitting (LOSO)
  - 17. Class Imbalance and Balancing
  - 18. Standardization
  - 19. Principal Component Analysis
  - 20. MinMax Scaling to Quantum Angles
  - 21. Quantum Machine Learning Architecture
    - 21.1 ZZFeatureMap
    - 21.2 RealAmplitudes Ansatz
    - 21.3 Variational Quantum Classifier (VQC)
    - 21.4 COBYLA Optimizer
  - 22. Training Pipeline
    - 22.1 Hyperparameter Search
    - 22.2 Checkpointing
    - 22.3 Training Time and Computational Cost
  - 23. Model Artifacts and Serialization
  - 24. Inference Pipeline
    - 24.1 Recording-Level Aggregation

## Appendix D. Detailed Fold Confusion Matrices

# 1. Executive Summary

Electroencephalography (EEG) is a non-invasive neuroimaging technique that records the electrical activity of the brain via electrodes placed on the scalp. EEG signals reflect the aggregate postsynaptic activity of millions of cortical neurons and are widely used in clinical neurology for diagnosing conditions such as epilepsy, sleep disorders, and brain injury. The signals are continuous, multi-channel, and highly variable between patients — making automated analysis both clinically valuable and technically challenging.

Epileptic seizures affect approximately 50 million people worldwide and represent one of the most common serious neurological conditions. A seizure is a transient occurrence of signs or symptoms resulting from abnormal, excessive, or synchronous neuronal activity in the brain. Manual interpretation of EEG recordings by trained neurologists is slow, labour-intensive, and subject to inter-rater variability. Automated seizure detection using machine learning offers the potential to assist clinicians in rapidly flagging suspicious recordings for expert review.

This project, the **Quantum EEG Signal Classifier**, implements a complete hybrid classical–quantum machine learning system for seizure detection. The system processes EEG recordings in European Data Format (EDF), extracts a 20-dimensional feature vector from each 2-second segment, applies dimensionality reduction via Principal Component Analysis (PCA) to produce 4 features, scales them to the range  $[0, \pi]$  for quantum angle encoding, and classifies each segment using a Variational Quantum Classifier (VQC) implemented in Qiskit. A React/TypeScript frontend allows users to upload EDF files and view segment-level probability charts, a detection timeline, and a detailed results table.

The system was trained and evaluated on a subset of the CHB-MIT Scalp EEG Database (subjects chb01–chb05). Evaluation uses nested Leave-One-Subject-Out (LOSO) cross-validation, which is the methodologically correct approach for EEG datasets where patient identity must not leak between training and test sets.

**■ Important:** The current production model (v1.3.0) achieves a mean accuracy of 91.19% but a mean F1-score of only 6.16% and a mean precision of 3.77%. These figures reflect a model that predominantly predicts non-seizure (majority class) and only partially detects seizures. The system is a research prototype and is NOT validated for clinical use. No quantum advantage has been demonstrated.

| Parameter             | Value                                |
|-----------------------|--------------------------------------|
| Model version         | v1.3.0                               |
| Classifier            | Variational Quantum Classifier (VQC) |
| Qubits                | 4                                    |
| Features (pre-PCA)    | 20 (4 channels × 5 features)         |
| Features (post-PCA)   | 4                                    |
| Dataset               | CHB-MIT chb01–chb05 (5 subjects)     |
| Mean Accuracy (LOSO)  | 91.19% ± 5.31%                       |
| Mean F1-Score (LOSO)  | 6.16% ± 4.97%                        |
| Mean Precision (LOSO) | 3.77% ± 2.95%                        |
| Mean Recall (LOSO)    | 18.29% ± 15.67%                      |
| Decision Threshold    | 0.60                                 |
| Training Duration     | ~60.6 minutes (3,637 seconds)        |

## 2. Problem Statement

---

### 2.1 EEG Signal Analysis

An EEG recording consists of multiple channels (electrode pairs), each recording a voltage time series sampled at a fixed rate (typically 256 Hz in the CHB-MIT dataset). Each channel captures local field potentials from a different brain region. Clinical EEG typically uses 18–64 channels following the 10-20 international system. The raw signals contain brain activity across multiple frequency bands: delta (0.5–4 Hz), theta (4–8 Hz), alpha (8–13 Hz), beta (13–30 Hz), and gamma (>30 Hz).

### 2.2 Seizure Detection Challenges

Automated seizure detection faces several fundamental challenges:

**Class Imbalance:** Seizure events occupy only a small fraction of total recording time. In the CHB-MIT dataset, seizure segments represent well under 5% of total segments. A naive classifier that predicts 'non-seizure' for every segment can achieve >95% accuracy while being clinically useless.

**Patient-to-Patient Variability:** EEG patterns differ substantially between individuals. A model trained on one patient's EEG may not generalise to another. This motivates subject-aware evaluation using LOSO cross-validation.

**High Dimensionality:** With multiple channels at 256 Hz, a 2-second segment contains  $4 \times 512 = 2,048$  raw data points. Classical and quantum models require dimensionality reduction before classification.

**Non-Stationarity:** EEG signals are highly non-stationary — statistical properties change over time. This complicates feature extraction and model generalisation.

**Artifact Contamination:** EEG is susceptible to motion artifacts, muscle artifacts, and electrode noise. Bandpass filtering attenuates some of these, but artifact removal is not fully implemented in this project.

### 2.3 Motivation for Hybrid Classical–Quantum ML

Quantum Machine Learning (QML) is an emerging field exploring whether quantum computing can enhance the performance of machine learning algorithms. A Variational Quantum Classifier (VQC) encodes classical feature values into quantum rotation angles, applies trainable quantum gate operations, measures the resulting quantum state, and uses the measurement output as a classification score. The primary motivation for exploring VQC in this project is research and educational: to demonstrate a complete hybrid classical–quantum pipeline on a real biomedical dataset.

■ **Note:** The current implementation runs the VQC on a classical quantum simulator (Qiskit statevector/Aer backend), not on physical quantum hardware. No quantum advantage has been demonstrated or claimed. The computational cost of VQC simulation is significantly higher than classical alternatives for the same problem size.

## 3. Project Objectives

---

### A. Primary Objectives (Implemented)

- Build a complete EEG signal processing pipeline: load EDF → validate → select channels → filter → segment → extract features.
- Train a Variational Quantum Classifier on CHB-MIT EEG data for seizure detection.
- Implement subject-aware LOSO cross-validation to prevent data leakage.
- Evaluate model performance using accuracy, precision, recall, F1, sensitivity, specificity.
- Serialize all model artifacts (VQC, PCA, scalers) for deployment-ready inference.
- Deploy a FastAPI REST API backend that loads saved artifacts and runs inference on uploaded EDF files.
- Build a React/TypeScript/Vite frontend with segment probability charts, detection timeline, and results table.

### B. Secondary Objectives (Implemented)

- Implement fold-level checkpointing to allow interrupted training to resume.
- Implement nested LOSO with inner validation for hyperparameter selection.
- Build a demo endpoint (/api/analyze-demo) for testing without user upload.
- Implement a DisclaimerBanner to clearly communicate non-clinical status.
- Implement an ExportMenu for downloading analysis results.
- Implement atomic artifact saving to prevent partial/corrupt checkpoint files.

### C. Research/Experimental Objectives (Partially Addressed)

- Explore whether VQC can achieve comparable performance to classical baselines (no baseline comparison implemented).
- Investigate effect of hyperparameters: PCA dimensions, class ratio, decision threshold.
- Investigate temporal post-processing experiments (diagnostic scripts present; not adopted in production).

### D. Deployment Objectives (Planned, Not Implemented)

Docker containerisation was planned (referenced in IMPLEMENTATION\_PLAN.md) but no Dockerfile or docker-compose.yml was found in the repository. Deployment is documented as planned but not confirmed as implemented.

## 4. System Requirements

### 4.1 Functional Requirements

| ID    | Requirement                                             | Status                           |
|-------|---------------------------------------------------------|----------------------------------|
| FR-01 | Accept EEG recordings in European Data Format (.edf)    | ✓ Implemented                    |
| FR-02 | Validate that required EEG channels are present         | ✓ Implemented                    |
| FR-03 | Validate sampling frequency and recording duration      | ✓ Implemented                    |
| FR-04 | Apply 0.5–40 Hz FIR bandpass filter                     | ✓ Implemented                    |
| FR-05 | Segment recording into 2-second non-overlapping windows | ✓ Implemented                    |
| FR-06 | Extract 20-dimensional feature vector per segment       | ✓ Implemented                    |
| FR-07 | Apply saved StandardScaler transform                    | ✓ Implemented                    |
| FR-08 | Apply saved PCA transform (20→4)                        | ✓ Implemented                    |
| FR-09 | Apply saved MinMaxScaler transform ([0,π])              | ✓ Implemented                    |
| FR-10 | Run saved VQC for segment-level classification          | ✓ Implemented                    |
| FR-11 | Apply decision threshold (0.60) to probabilities        | ✓ Implemented                    |
| FR-12 | Aggregate segment predictions to recording-level result | ✓ Implemented                    |
| FR-13 | Return structured API response with all predictions     | ✓ Implemented                    |
| FR-14 | Display segment probability chart in frontend           | ✓ Implemented                    |
| FR-15 | Display detection timeline in frontend                  | ✓ Implemented                    |
| FR-16 | Export results to JSON/CSV                              | ~ Partially (ExportMenu present) |
| FR-17 | Docker containerised deployment                         | ✗ Not implemented                |

### 4.2 Non-Functional Requirements

| Category        | Requirement                                                    | Implementation                                                         |
|-----------------|----------------------------------------------------------------|------------------------------------------------------------------------|
| Reproducibility | Preprocessing must be identical between training and inference | Shared eeg_processing/pipeline.py used by both                         |
| Data Integrity  | No fitting of transformers during inference                    | InferenceService calls only .transform()                               |
| Modularity      | Each processing step in its own module                         | 9 modules in eeg_processing/; separate training/, quantum/, inference/ |
| Error Handling  | Invalid EDF must raise informative exception                   | EdfFileNotFoundError, EdfChannelError, etc. in core/exceptions.py      |
| Consistency     | Feature schema hash ensures training/inference compatibility   | SHA-256 hash stored in feature_config.json                             |
| Performance     | Stateless InferenceService loads artifacts once                | Singleton pattern in api/main.py startup                               |
| Usability       | User sees processing state and error details                   | ProcessingOverlay and ERROR app state in App.tsx                       |
| Auditability    | Model metadata records training subjects, config, metrics      | model_metadata.json persisted per model version                        |

### 4.3 Software Requirements

| Component | Requirement           | Specified Version |
|-----------|-----------------------|-------------------|
| Python    | Core backend language | >=3.11            |

|                         |                                |                          |
|-------------------------|--------------------------------|--------------------------|
| Qiskit                  | Quantum circuit library        | >=2.0                    |
| Qiskit Machine Learning | VQC implementation             | >=0.9.0                  |
| MNE                     | EEG/EDF processing             | >=1.6                    |
| NumPy                   | Numerical computing            | >=1.26                   |
| SciPy                   | Signal processing (Welch PSD)  | >=1.12                   |
| PyWavelets              | Wavelet decomposition          | >=1.5                    |
| scikit-learn            | PCA, scalars, metrics          | >=1.4                    |
| pandas                  | Data manipulation              | >=2.1                    |
| FastAPI                 | REST API backend               | Latest compatible        |
| dill                    | VQC serialization              | >=0.3.8                  |
| joblib                  | Sklearn artifact serialization | >=1.3                    |
| Node.js / npm           | Frontend build tooling         | Compatible with Vite 5.x |
| React                   | Frontend library               | ^18.2.0                  |
| TypeScript              | Frontend language              | ^5.2.2                   |
| Vite                    | Frontend build tool            | ^5.1.6                   |
| Tailwind CSS            | Frontend styling               | ^3.4.1                   |
| Recharts                | Data visualization             | ^2.12.2                  |
| Lucide React            | Icon library                   | ^0.344.0                 |

## 5. Technology Stack

---

### Python 3.11+

**Category:** Core language | **Used in:** All backend modules

Type hints, dataclasses, pathlib, match statements; Python 3.11 is required for performance improvements and modern syntax used throughout the codebase.

### MNE (>=1.6)

**Category:** EEG/EDF processing | **Used in:** eeg\_processing/loader.py, validator.py, channel\_selector.py, filtering.py

MNE provides a mature, well-tested EDF parser (`mne.io.read_raw_edf`), supports lazy header-only loading, and implements FIR bandpass filtering with correct phase handling. Manually parsing EDF would require implementing the EDF specification — MNE eliminates this complexity.

### PyWavelets (>=1.5)

**Category:** Wavelet decomposition | **Used in:** eeg\_processing/feature\_extraction.py

Implements Discrete Wavelet Transform (DWT) via `pywt.wavedec()`. Used to extract approximation coefficients (low frequency, delta/theta) and detail coefficients (alpha/low-beta) as features.

### SciPy (>=1.12)

**Category:** Signal processing | **Used in:** eeg\_processing/feature\_extraction.py

Provides `scipy.signal.welch()` for Power Spectral Density estimation used in the delta band power feature calculation.

### scikit-learn (>=1.4)

**Category:** ML utilities | **Used in:** training/prepare\_dataset.py, evaluate.py

Provides StandardScaler, PCA, MinMaxScaler, and classification metrics (`accuracy_score`, `precision_score`, etc.). Mature, well-tested, and widely adopted.

### Qiskit (>=2.0)

**Category:** Quantum computing | **Used in:** quantum/classifier.py

Provides ZZFeatureMap, RealAmplitudes quantum circuit components. The primary quantum SDK for this project.

### Qiskit Machine Learning (>=0.9)

**Category:** VQC algorithm | **Used in:** quantum/classifier.py

Provides the high-level VQC class that wraps ZZFeatureMap + ansatz + optimizer into a scikit-learn-compatible classifier interface.

### FastAPI

**Category:** REST API | **Used in:** api/main.py

Async-capable Python web framework with automatic OpenAPI documentation, request validation, and file upload handling. Chosen over Flask for modern async support and type-safety via Pydantic.

### dill (>=0.3.8)

**Category:** VQC serialization | **Used in:** training/save\_artifacts.py, checkpoint.py

The standard pickle module cannot serialize Qiskit circuit objects (which contain lambda functions and custom C extensions). dill extends pickle to handle these objects. Used specifically for vqc\_model.dill.

### joblib (>=1.3)

**Category:** sklearn serialization | **Used in:** training/save\_artifacts.py

Efficient serialization optimized for NumPy arrays (memory-mapped files). Used for pre\_scaler.joblib, pca.joblib, scaler.joblib — all of which contain NumPy arrays.

### React 18

**Category:** Frontend UI | **Used in:** frontend/src/

Component-based UI library enabling separation of each visual element (chart, table, timeline) into independently managed components. React's hooks (useState, useEffect) manage the application state machine.

### TypeScript 5.2

**Category:** Frontend language | **Used in:** frontend/src/

Provides compile-time type checking for API response shapes, preventing runtime errors when accessing result.segment\_predictions or result.seizure\_probability.

### Vite 5

**Category:** Frontend build | **Used in:** frontend/

Fast HMR development server and optimised production build tool. Configured with @vitejs/plugin-react.

### Tailwind CSS 3.4

**Category:** Frontend styling | **Used in:** frontend/src/

Utility-first CSS framework providing consistent spacing, colour, and responsive design without writing custom CSS.

### Recharts 2.12

**Category:** Data charts | **Used in:** ProbabilityChart.tsx, DetectionTimeline.tsx

React-native chart library based on D3. Provides AreaChart, LineChart, BarChart components used for the segment probability visualization.

### Lucide React 0.344

**Category:** Icons | **Used in:** Multiple components

Provides consistent icon set (AlertCircle, Upload, RefreshCw, etc.) used throughout the UI.



## 6. Complete System Architecture

### 6.1 High-Level System Architecture

Figure 1. High-Level System Architecture

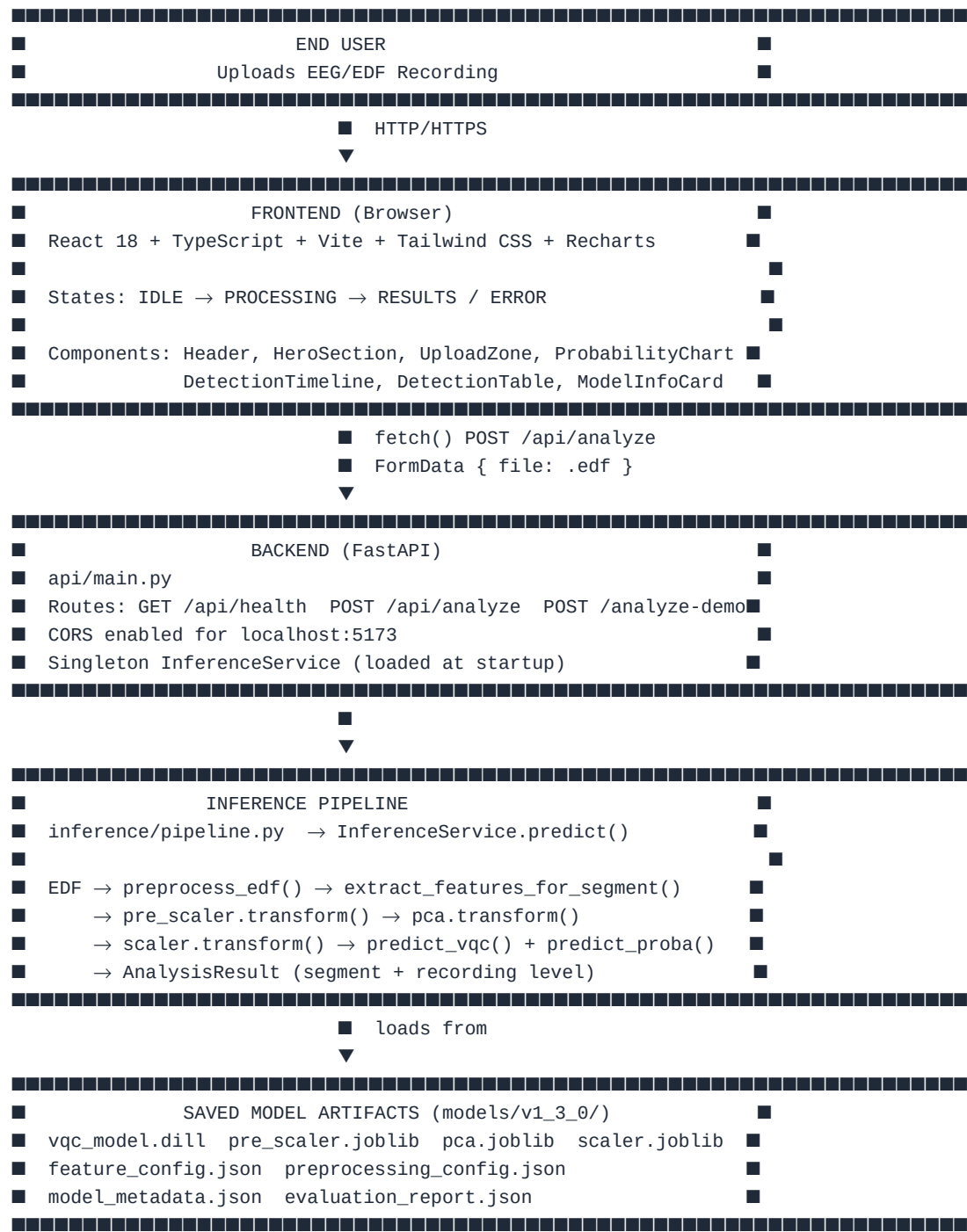


Figure 1. The complete runtime system architecture showing the user-facing frontend, FastAPI backend, inference pipeline, and saved model artifacts.

### 6.2 Training Architecture (Offline)

### Figure 2. Training Pipeline Architecture



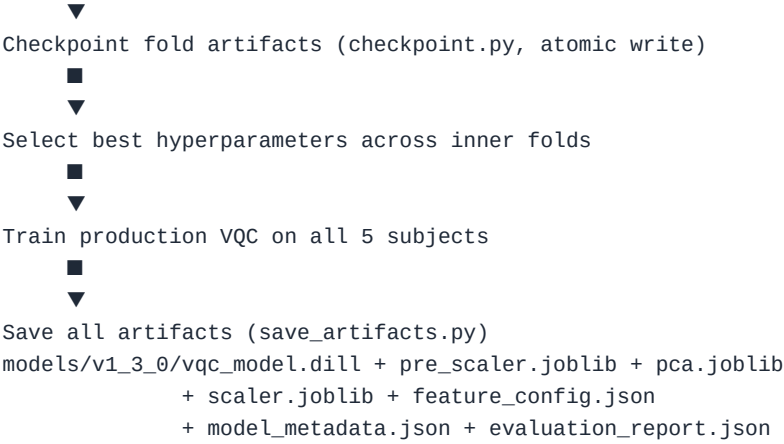


Figure 2. Complete offline training pipeline from raw EDF files to saved model artifacts.

6.3 Critical Distinction: Training vs Inference

This is one of the most important architectural principles in the system. The entire project follows the paradigm: **TRAIN ONCE** → **SAVE ARTIFACTS** → **DEPLOY FOR INFERENCE**.

| Step           | Training                             | Inference                               |
|----------------|--------------------------------------|-----------------------------------------|
| StandardScaler | fit(X_train) then transform()        | transform() ONLY — using saved scaler   |
| PCA            | fit(X_train_scaled) then transform() | transform() ONLY — using saved PCA      |
| MinMaxScaler   | fit(X_train_pca) then transform()    | transform() ONLY — using saved scaler   |
| VQC            | vqc.fit(X_train, y_train)            | predict_proba() ONLY — no weight update |
| Labels         | Required (y_train, y_test)           | Not required — prediction only          |
| CHB-MIT data   | Required                             | NOT required                            |

WHY this matters — **Data Leakage**: If the StandardScaler, PCA, or MinMaxScaler were re-fitted on the user's uploaded EDF file during inference, the resulting transformed values would be normalised according to the statistics of that specific recording. This would break the mapping learned during training, because the VQC was trained on data normalised according to the training set statistics. The system would be applying a different normalisation to inference data than was used during training, invalidating all predictions. The saved transformers preserve the exact scaling learned from the training set.

## 7. EEG Dataset — CHB-MIT

The CHB-MIT Scalp EEG Database is a publicly available dataset collected at Boston Children's Hospital (Shoeb, 2009) and distributed via PhysioNet. It contains scalp EEG recordings from 23 paediatric patients with intractable epilepsy, collected over multiple sessions. Recordings are stored in European Data Format (EDF), a widely adopted standard for physiological time-series data.

### 7.1 EDF Format

EDF (European Data Format) is a binary file format for multichannel physiological recordings. Each EDF file contains: a header with channel names, sampling frequencies, physical/digital range, and signal information; followed by data records containing the actual time-series samples. MNE's `mne.io.read_raw_edf()` handles header parsing and data access transparently.

### 7.2 Seizure Annotations

The CHB-MIT dataset includes per-subject `*-summary.txt` files that describe each EDF recording and list seizure onset and offset times in seconds. For example:

|                                  |
|----------------------------------|
| File Name: chb01_03.edf          |
| Number of Seizures in File: 1    |
| Seizure Start Time: 2996 seconds |
| Seizure End Time: 3036 seconds   |

This project parses these summary files using regular expressions (`eeg_processing/annotations.py`) to extract seizure intervals for each EDF file. Note: The binary `.seizures` sidecar files are NOT used; the summary `.txt` files are the authoritative annotation source.

### 7.3 Dataset Subset Used in This Project

| Parameter                    | Full CHB-MIT Dataset   | This Project's Subset                   |
|------------------------------|------------------------|-----------------------------------------|
| Subjects                     | 23 patients            | 5 patients (chb01–chb05)                |
| Recordings                   | ~664 EDF files         | Subset of chb01–chb05 recordings        |
| Sampling Rate                | 256 Hz (most patients) | 256 Hz                                  |
| Channels                     | 23 (typically)         | 4 selected: FP1-F7, F3-C3, FZ-CZ, P4-O2 |
| Total Segments (v1.3.0 eval) | Not specified          | 56,820 segments across 5 LOSO folds     |
| Training Segments            | Not specified          | 1,972 balanced samples (production VQC) |

■ **Note:** Only 5 subjects were used in this project. The evaluation report shows total test segments across all 5 folds:  $11,962 + 4,079 + 12,600 + 19,179 + 9,000 = 56,820$ . This represents a small subset of the complete CHB-MIT dataset.

## 8. EEG Data Loading — loader.py

### 8.1 File: eeg\_processing/loader.py

**Purpose:** Load EDF files using MNE and extract metadata. Provides two functions: `load_edf()` and `extract_metadata()`.

**Why MNE?** MNE is the de facto standard Python library for EEG/MEG data processing. It correctly implements EDF header parsing, handles different EDF variants (EDF, EDF+, BDF), provides lazy loading (reading only the header without loading all data into memory), and supports subsequent operations like channel selection and filtering on the same Raw object. Writing a custom EDF parser would require implementing the EDF byte specification — error-prone and unnecessary when MNE exists.

```
# eeg_processing/loader.py – load_edf()
raw = mne.io.read_raw_edf(str(file_path), preload=preload, verbose=False)
```

**preload=False** (the default): MNE reads only the EDF header. The data is not loaded into RAM. This is memory-efficient for large recordings and allows the validator to inspect channel names and sampling rate without loading gigabytes of signal data. Data is loaded later when `bandpass_filter()` calls `raw.copy().load_data()`.

**verbose=False:** Suppresses MNE's extensive console output during batch processing.

**Input:** Path to .edf file. **Output:** `mne.io.Raw` object (header loaded, data lazy). **Errors raised:** `EdfFileNotFoundError` if file missing; `EdfInvalidFormatError` if MNE cannot parse the file.

## 9. EEG Validation — validator.py

Before any processing begins, the recording is validated to ensure it meets the requirements of the trained model. Three checks are performed:

| Validation    | Function                              | Checked Condition                                          | Exception if Failed               |
|---------------|---------------------------------------|------------------------------------------------------------|-----------------------------------|
| Channel check | <code>validate_channels()</code>      | All 4 required channels present (case-insensitive)         | <code>EdfChannelError</code>      |
| Sampling rate | <code>validate_sampling_rate()</code> | <code>sfreq &gt; 0</code> (and optional: matches expected) | <code>EdfSamplingRateError</code> |
| Duration      | <code>validate_duration()</code>      | Recording <code>&gt;= 2.0</code> seconds                   | <code>EdfDurationError</code>     |

**Why validate channels?** The trained VQC was trained on exactly 4 specific channels in a fixed order. If an uploaded EDF is missing one of these channels, or has them in a different order, the resulting feature vector would be meaningless — the VQC would be receiving features from the wrong brain regions. Channel validation prevents this failure mode and returns a clear error message to the user.

```
# validator.py – validate_channels()
available = {ch.strip().upper(): ch for ch in raw.ch_names}
missing = [ch for ch in required if ch not in available]
if missing:
    raise EdfChannelError(missing_channels=missing_original, ...)
```

Channel names are normalized (stripped + uppercased) before comparison to handle minor formatting differences in EDF headers (e.g., 'fp1-f7' vs 'FP1-F7').

## 10. Channel Selection — channel\_selector.py

After validation confirms all required channels are present, only those 4 channels are selected and reordered. The remaining channels are discarded.

| Channel | Brain Region                       | Approximate Location       |
|---------|------------------------------------|----------------------------|
| FP1-F7  | Left frontopolar/frontal           | Anterior left hemisphere   |
| F3-C3   | Left frontocentral                 | Central left hemisphere    |
| FZ-CZ   | Frontal midline to central midline | Medial/central scalp       |
| P4-O2   | Right parieto-occipital            | Posterior right hemisphere |

**Why only 4 channels?** Using fewer channels reduces the feature vector size, decreases computational cost (especially critical for quantum circuits where each qubit corresponds to one post-PCA feature), and improves consistency between training recordings and user-uploaded recordings (which must have the same channels). The choice of these specific 4 channels was a design decision rather than one experimentally optimised for seizure detection.

■ **Note:** The 4-channel selection was not experimentally validated against other channel combinations. No clinical optimality is claimed. The channels cover multiple brain regions (frontal, central, parieto-occipital) to provide spatial diversity.

```
# channel_selector.py — select_channels()
raw_selected = raw.copy().pick(pick_names)
```

`raw.copy()` creates a new Raw object so the original is not modified. `.pick()` selects and reorders channels in one operation. The resulting Raw object contains exactly 4 channels in the order: FP1-F7, F3-C3, FZ-CZ, P4-O2.

## 11. Signal Filtering — filtering.py

A Finite Impulse Response (FIR) bandpass filter is applied across all selected channels. The filter passes frequencies between 0.5 Hz and 40.0 Hz and attenuates all others.

```
# filtering.py — bandpass_filter()
raw_filtered = raw.copy().load_data()
raw_filtered.filter(l_freq=0.5, h_freq=40.0, method='fir', verbose=False)
```

**Why 0.5 Hz lower bound?** Frequencies below 0.5 Hz include DC offset and slow baseline drift (caused by electrode movement, sweat, or amplifier drift). These artifactual components are not neurologically meaningful and would corrupt features like Hjorth mobility and spectral power.

**Why 40.0 Hz upper bound?** The clinically relevant EEG bands for seizure detection are delta (0.5–4), theta (4–8), alpha (8–13), and beta (13–30) Hz. Frequencies above 40 Hz are dominated by muscle artifact (EMG), mains noise (50/60 Hz), and high-frequency gamma activity. Cutting at 40 Hz retains the clinically relevant information while removing most artifact.

**Why FIR?** MNE's default `method='fir'` applies a linear-phase FIR filter, which introduces no phase distortion and does not shift event timings — important when aligning seizure onset times with filtered segments.

**load\_data():** Calling `.load_data()` before filtering forces the data from lazy/memory-mapped mode into RAM — necessary before in-place filtering.

**Why apply the same filter at inference?** The feature extractor (wavelet, spectral, entropy features) was developed and trained on filtered data. If inference data were not filtered, the feature values would differ

systematically from training feature values — breaking the learned decision boundary.

## 12. Segmentation — segmentation.py

**Figure 3. EEG Segmentation**

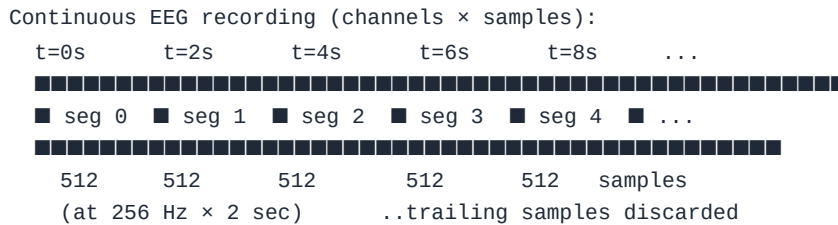


Figure 3. Non-overlapping 2-second segmentation of a continuous EEG recording at 256 Hz.

```
# segmentation.py – segment_recording()
samples_per_segment = int(sfreq * segment_duration_seconds)
n_segments = n_total_samples // samples_per_segment
for i in range(n_segments):
    start = i * samples_per_segment
    end = start + samples_per_segment
    segments.append(EEGSegment(
        index=i, start_seconds=start/sfreq, end_seconds=end/sfreq,
        data=data[:, start:end], # (n_channels, samples_per_segment)
    ))
```

**Why 2-second segments?** Two seconds is long enough to capture multiple cycles of the lowest EEG frequency band of interest (0.5 Hz = 2 sec period; delta band 0.5–4 Hz). It is short enough that a single segment either overlaps a seizure event or does not — enabling meaningful binary labeling. Shorter segments (e.g., 0.5 sec) risk insufficient spectral resolution for delta band features. Longer segments (e.g., 10 sec) may straddle seizure onset, complicating labeling.

**Why non-overlapping?** Overlapping segments would create correlated samples — adjacent segments sharing most of their data. This correlation inflates apparent model performance (information leaks between training examples). Non-overlapping segments are statistically more independent.

**At 256 Hz:**  $\text{samples\_per\_segment} = \text{int}(256 \times 2.0) = 512$  samples per segment per channel. Each EEGSegment.data has shape (4, 512).

**Trailing samples:** If  $\text{total\_samples} \% 512 \neq 0$ , the final incomplete window is discarded. This is the correct behaviour — an incomplete segment would have fewer than 512 samples and would produce incorrect feature values.

## 13. Seizure Annotation and Labeling — annotations.py, build\_dataset.py

### 13.1 Annotation Parsing (annotations.py)

The CHB-MIT summary.txt files are parsed using regular expressions to extract seizure intervals for each EDF file.

```
# annotations.py – parse_summary_file()
```

```

starts = re.findall(
    r'Seizure(?:\s+\d+)?\s+Start\s+Time:\s*(\d+)\s*seconds', block
)
ends = re.findall(
    r'Seizure(?:\s+\d+)?\s+End\s+Time:\s*(\d+)\s*seconds', block
)
for s, e in zip(starts, ends):
    intervals.append(SeizureInterval(
        start_seconds=float(s), end_seconds=float(e))

```

The regex pattern handles both 'Seizure Start Time:' and 'Seizure N Start Time:' variants that appear in different CHB-MIT subjects. If starts and ends counts differ, a warning is logged and mismatched pairs are skipped.

## 13.2 Segment Labeling (build\_dataset.py)

Each 2-second segment is labeled based on whether it overlaps with any seizure interval:

```

# build_dataset.py - _label_segment()
def _label_segment(segment, seizure_intervals):
    seg_start = segment.start_seconds
    seg_end = segment.end_seconds
    for interval in seizure_intervals:
        # Two ranges [a,b) and [c,d) overlap iff a<d AND c<b
        if seg_start < interval.end_seconds and \
            interval.start_seconds < seg_end:
            return Label.SEIZURE # 1
    return Label.NON_SEIZURE # 0

```

**Overlap condition:** Two intervals  $[a, b)$  and  $[c, d)$  overlap if and only if  $a < d$  AND  $c < b$ . This is the standard interval overlap test. Even a single sample of overlap triggers a seizure label. This is intentionally conservative — if any part of a 2-second window falls within a seizure interval, the window is considered a seizure segment.

**Why any-overlap labeling?** Because seizure onset often occurs mid-segment, waiting for full containment would miss the first and last seizure segment. The alternative (strict containment) would systematically miss seizure boundaries.

## 14. Feature Extraction — feature\_extraction.py

Feature extraction converts each 2-second EEG segment (shape: 4 channels × 512 samples) into a compact 20-dimensional feature vector. Five features are computed for each of the 4 channels, yielding  $4 \times 5 = 20$  features per segment.

### Figure 4. Feature Vector Construction

```
Channel FP1-F7 (512 samples) → [f1, f2, f3, f4, f5]
Channel F3-C3 (512 samples) → [f1, f2, f3, f4, f5]
Channel FZ-CZ (512 samples) → [f1, f2, f3, f4, f5]
Channel P4-O2 (512 samples) → [f1, f2, f3, f4, f5]
    ■
    ▼ concatenate
20-dimensional feature vector ∈ R^20
f1 = wavelet_approx_energy (coeffs[0], DWT level 4, db4)
f2 = wavelet_energy        (coeffs[1], DWT level 4, db4)
f3 = shannon_entropy       (histogram-based)
f4 = delta_band_power      (Welch PSD, 0.5–4.0 Hz relative power)
f5 = hjorth_mobility       (std(diff(x)) / std(x))
```

Figure 4. Construction of the 20-dimensional feature vector from 4 EEG channels.

### 14.1 Feature 1: Wavelet Approximation Energy

Computed using `pywt.wavedec()` with Daubechies 4 (db4) wavelet at decomposition level 4.

```
# feature_extraction.py — _wavelet_approx_energy()
coeffs = pywt.wavedec(segment, 'db4', level=4)
approx = coeffs[0] # approximation: lowest-frequency band
return float(np.sum(approx ** 2))
```

**What it measures:** The total energy in the approximation (lowest-frequency) sub-band of the wavelet decomposition. For db4 at level 4 and  $fs=256$  Hz, the approximation coefficients correspond approximately to the 0–8 Hz band (delta and theta brain activity).

**Why coeffs[0]?** `pywt.wavedec()` returns coefficients in order: [cA4, cD4, cD3, cD2, cD1], where cA4 is the level-4 approximation (lowest frequency) and cD4, cD3... are successive detail levels (higher frequencies). `coeffs[0] = cA4` is correct for the approximation band.

**Why useful for seizure detection?** Seizures are often characterised by high-amplitude rhythmic activity in the delta/theta band. Increased approximation energy can indicate the onset of synchronised slow-wave activity. The docstring reports Cohen's  $d = 0.974$  as an empirical discriminability measure for this feature.

### 14.2 Feature 2: Wavelet Detail Energy

```
# feature_extraction.py — _wavelet_energy()
coeffs = pywt.wavedec(segment, 'db4', level=4)
detail = coeffs[1] # level-4 detail: ~8-16 Hz (alpha/low-beta)
return float(np.sum(detail ** 2))
```

**What it measures:** Energy in the level-4 detail coefficients (cD4), corresponding approximately to the 8–16 Hz band (alpha and low-beta activity). Changes in alpha suppression and beta activity can accompany seizures.

■ **Important:** IMPLEMENTATION AUDIT NOTE: This feature is named 'wavelet\_energy' in the feature configuration. The docstring says '8–16 Hz alpha/low-beta'. `coeffs[1] = cD4` (level-4 detail) is the correct band.

The code is consistent with what is documented in the comments. No mismatch was detected between intended and actual behaviour for this feature.

### 14.3 Feature 3: Shannon Entropy

```
# feature_extraction.py - _shannon_entropy()
hist, _ = np.histogram(segment, bins=64, density=True)
hist = hist[hist > 0]
p = hist / (hist.sum() + 1e-12)
return float(-np.sum(p * np.log2(p + 1e-12)))
```

**What it measures:** The entropy of the amplitude distribution of the EEG segment. High entropy = more uniform/irregular signal distribution. Low entropy = more structured/predictable distribution. Seizures are characterised by highly regular, synchronised oscillations which manifest as lower entropy compared to background EEG which is more irregular.

**Implementation:** A 64-bin amplitude histogram estimates the probability distribution  $P(\text{amplitude})$ . Shannon entropy  $H = -\sum p(x) \log p(x)$  is computed over the non-zero bins. The epsilon (1e-12) prevents  $\log(0)$ .

### 14.4 Feature 4: Delta Band Power

```
# feature_extraction.py - _delta_band_power()
freqs, psd = sp_signal.welch(segment, fs=sfreq, nperseg=min(len(segment), int(sfreq*2)))
total_power = trapz(psd, freqs) + 1e-12
delta_mask = (freqs >= 0.5) & (freqs <= 4.0)
delta_power = trapz(psd[delta_mask], freqs[delta_mask])
return float(delta_power / total_power)
```

**What it measures:** The relative power in the delta band (0.5–4.0 Hz) as a fraction of total signal power. Computed via Welch's method for robust Power Spectral Density (PSD) estimation. Delta band power is typically elevated during seizures, especially post-ictally (after seizure activity).

**Relative vs absolute power:** Using relative power (fraction of total) reduces sensitivity to overall signal amplitude differences between recordings and electrodes.

### 14.5 Feature 5: Hjorth Mobility

```
# feature_extraction.py - _hjorth_mobility()
dx = np.diff(segment) # first derivative approximation
std_x = np.std(segment) + 1e-12
std_dx = np.std(dx)
return float(std_dx / std_x)
```

**What it measures:** Hjorth parameters are time-domain complexity measures. Mobility =  $\text{std}(dx/dt) / \text{std}(x)$ , estimating the mean frequency of the signal. Higher mobility = higher-frequency content. During seizures, the frequency content of EEG often increases significantly, making Hjorth Mobility a discriminative feature.

**Why engineered features instead of raw EEG into VQC?** The VQC has 4 qubits corresponding to 4 post-PCA features. Feeding raw EEG directly would require  $512 \text{ samples} \times 4 \text{ channels} = 2048$  qubits — far beyond current quantum hardware or simulation capability. Engineered features + PCA compress the signal to 4 dimensions suitable for a 4-qubit circuit.

### 14.6 Docstring Inconsistency Found

■ **Important:** The function `extract_features_for_segment()` contains a docstring that incorrectly states '16-dimensional feature vector' in its Returns section (line 130–131 of `feature_extraction.py`). The actual implementation produces a 20-dimensional vector (4 channels × 5 features). The code itself is correct — `expected_dim = config.total_features = 20` — but the docstring reflects an earlier version of the feature set. This is a documentation-code inconsistency.

## 15. Dataset Construction — build\_dataset.py

The `build_dataset()` function orchestrates the entire training data construction: subject discovery → annotation parsing → EDF preprocessing → feature extraction → labeling → dataset assembly.

```
# build_dataset.py — build_dataset() structure
for subject_id in subjects:
    annotations = parse_summary_file(summary_path)
    for edf_path in edf_files:
        segments, sfreq = preprocess_edf(edf_path, feature_config)
        seizure_intervals = get_seizure_intervals(annotations, edf_path.name)
        for seg in segments:
            label = _label_segment(seg, seizure_intervals)
            feat = extract_features_for_segment(seg.data, sfreq, feature_config)
            all_X.append(feat) # 20-dim feature vector
            all_y.append(label) # 0 or 1
            all_meta.append(SegmentMeta(subject_id, recording_id, ...))
X = np.vstack(all_X) # shape: (N_segments, 20)
y = np.array(all_y) # shape: (N_segments,)
```

**Error handling:** If an EDF file fails to load or feature extraction fails for a segment, the error is caught, logged, and the file/segment is added to the skipped list. This prevents a single bad recording from aborting the entire dataset build. The `DatasetResult.skipped` field records all skipped files and reasons.

**Metadata retention:** Each segment is accompanied by a `SegmentMeta` object recording `subject_id`, `recording_id`, `segment_index`, `start/end seconds`, and `label`. This is essential for LOSO splitting (grouping by `subject_id`) and for audit/debugging.

## 16. Subject-Aware Data Splitting — LOSO

Figure 5. Leave-One-Subject-Out (LOSO) Cross-Validation

5 Subjects: chb01 chb02 chb03 chb04 chb05

Fold 0: TEST = chb01 TRAIN = chb02, chb03, chb04, chb05

Fold 1: TEST = chb02 TRAIN = chb01, chb03, chb04, chb05

Fold 2: TEST = chb03 TRAIN = chb01, chb02, chb04, chb05

Fold 3: TEST = chb04 TRAIN = chb01, chb02, chb03, chb05

Fold 4: TEST = chb05 TRAIN = chb01, chb02, chb03, chb04

→ Each subject is test subject exactly once

→ No segment from the test subject is ever in the training set

→ Preprocessing (StandardScaler, PCA, MinMaxScaler) fitted on TRAIN only

→ VQC fitted on TRAIN only

→ Evaluation performed on TEST only

Figure 5. LOSO cross-validation with 5 subjects. Each fold holds one subject out as the test set.

**Why LOSO?** Random splitting of segments would allow segments from the same patient to appear in both training and test sets. Since EEG patterns are highly patient-specific, a model trained on some segments from patient X would 'recognise' other segments from patient X in the test set — inflating apparent generalisation. LOSO ensures each test fold contains an entirely unseen patient, measuring true cross-patient generalisation performance.

**Nested LOSO for hyperparameter selection:** The evaluation report documents 'nested\_losos\_with\_rotating\_inner\_validation'. For each outer LOSO fold (one subject held out as test), the training subjects are further split in an inner validation scheme to select hyperparameters (class ratio, threshold). This prevents the hyperparameter search from using information from the test subject.

## 17. Class Imbalance and Balancing

In the CHB-MIT dataset, seizure events occupy only a small fraction of recording time. This creates a severe class imbalance where non-seizure segments vastly outnumber seizure segments. A model trained on imbalanced data tends to predict the majority class (non-seizure) for most inputs, achieving high accuracy but poor seizure detection.

**v1.3.0 strategy:** Training-only undersampling of the majority class (non-seizure) to achieve a 1:1 ratio (class\_ratio = 1.0). This means for N seizure segments, N non-seizure segments are randomly selected (seed=42 for reproducibility) to form the balanced training set.

**The test set is NOT balanced.** The test set retains its natural class distribution. This is methodologically correct: balancing the test set would artificially inflate seizure detection metrics by removing the difficulty of detecting rare events in realistic conditions.

**Why class\_ratio = 1.0?** The hyperparameter search tested ratios [1.0, 2.0]. The inner validation selected ratio 1.0 as optimal across all 5 folds. Ratio 1.0 means equal numbers of seizure and non-seizure samples in training.

**Production training used 1,972 balanced samples** (as recorded in evaluation\_report.json: production\_balanced\_samples = 1,972).

## 18. Standardization — StandardScaler

```
# prepare_dataset.py — fit_pre_scaler()
scaler = StandardScaler()
scaler.fit(X_train) # computes mean and std for each of 20 features
# Applied as: x_scaled = (x - mean) / std
```

**What it does:** StandardScaler transforms each feature to have zero mean and unit variance across the training set:  $z = (x - \mu) / \sigma$ .

**Why apply before PCA?** PCA is sensitive to feature scale — a feature with large variance (e.g., wavelet energy in microvolts<sup>2</sup>) would dominate the principal components over features with small variance (e.g., Shannon entropy in bits). StandardScaler ensures all 20 features contribute equally to the PCA decomposition.

**Why fit only on training data?** Fitting on the test set or on user-uploaded data would use test-set statistics to normalise — a form of data leakage. The saved pre\_scaler.joblib contains the training-set mean and std vectors and is applied as-is during inference.

## 19. Principal Component Analysis

| PC Component         | Explained Variance Ratio | Cumulative |
|----------------------|--------------------------|------------|
| PC1                  | 35.68%                   | 35.68%     |
| PC2                  | 20.86%                   | 56.54%     |
| PC3                  | 10.91%                   | 67.44%     |
| PC4                  | 5.02%                    | 72.46%     |
| PCs 5–20 (discarded) | 27.54%                   | 100.00%    |

Table: PCA explained variance ratios from model\_metadata.json (v1.3.0).

**What PCA does:** Principal Component Analysis finds the linear projections of the 20-dimensional feature space that capture the most variance. The top 4 principal components capture 72.46% of the total variance in the training features.

**Why PCA to 4 components?** This is driven by the quantum circuit constraint: the VQC has 4 qubits, requiring exactly 4 input features (one quantum angle per qubit). PCA provides a principled dimensionality reduction from 20  $\rightarrow$  4 that preserves maximum variance. The hyperparameter search tested `pca_dims = [4]` (only one dimension was explored in v1.3.0).

**Limitation:** 27.54% of feature variance is discarded by PCA. This is information loss. It is possible that some discriminative features for seizure detection are in the discarded components. A future improvement would be to test more PCA dimensions (e.g., 6 or 8 with corresponding qubits).

## 20. MinMax Scaling to Quantum Angles

After PCA reduces the features to 4 dimensions, a `MinMaxScaler` maps each dimension to the range  $[0, \pi]$  (approximately  $[0.0, 3.14159]$ ).

```
# prepare_dataset.py - fit_scaler()
scaler = MinMaxScaler(feature_range=(0.0, 3.14159265358979))
scaler.fit(X_train_pca)
# Applied as: x_scaled = (x - x_min) / (x_max - x_min) *  $\pi$ 
```

**Why  $[0, \pi]$ ?** The `ZZFeatureMap` encodes classical features as quantum rotation angles in  $R_y(\theta)$  rotation gates. Quantum rotation gates have a natural period of  $2\pi$ . Mapping features to  $[0, \pi]$  ensures each feature covers half the full rotation range — sufficient to span the meaningful part of the quantum feature space without wrapping around (which would make large and small values indistinguishable).

**Why `MinMaxScaler` after `StandardScaler`?** `StandardScaler` normalises feature distributions to zero mean / unit variance (output range is unbounded, roughly  $[-3, +3]$ ). `MinMaxScaler` then linearly maps this bounded range to  $[0, \pi]$ . The two-stage approach handles outliers better than applying `MinMaxScaler` directly to raw features.

## 21. Quantum Machine Learning Architecture

**Figure 6. Conceptual Quantum Circuit Architecture (VQC)**

Classical 4-dim Feature Vector (scaled to  $[0, \pi]$ ):

$[x_0, x_1, x_2, x_3]$

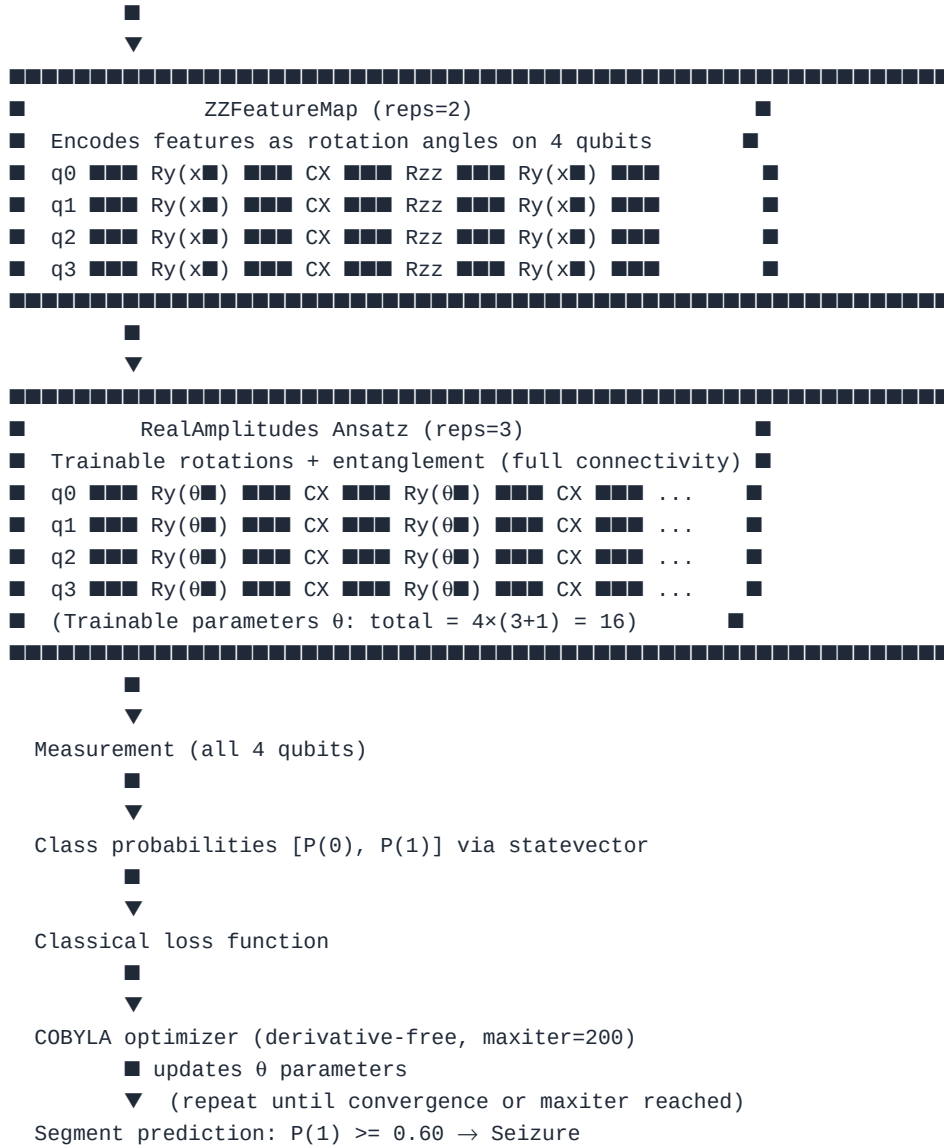


Figure 6. Conceptual representation of the VQC. Actual gate sequence depends on Qiskit transpilation.

### 21.1 ZZFeatureMap

**What it is:** A parameterised quantum circuit that encodes classical feature values into quantum states. ZZFeatureMap uses Hadamard gates (H), Pauli-X rotations (Rx), and ZZ interaction gates (CX-Rz-CX patterns) to create an entangled quantum state that depends on all input feature values and their pairwise interactions.

**feature\_dimension=4:** Matches the 4 post-PCA, post-MinMax-scaled features. Each feature value  $x_i \in [0, \pi]$  is used as a rotation angle.

**reps=2:** The encoding layer is applied twice. More repetitions create deeper circuits with more expressive feature encoding, but also increase computational cost and risk barren plateaus during training.

**entanglement='full':** Every qubit pair is connected by entangling operations. This enables the circuit to capture pairwise correlations between all input features in the quantum representation.

**Why ZZFeatureMap?** It is the standard feature map for QML with Qiskit ML and includes both single-feature encoding (diagonal terms) and two-feature interaction encoding (off-diagonal ZZ terms). The ZZ interaction is believed to capture correlations between features that may be relevant for classification.

## 21.2 RealAmplitudes Ansatz

**What it is:** The 'ansatz' is the trainable part of the VQC circuit. RealAmplitudes uses only Ry (Y-axis rotation) gates and CNOT entangling gates, keeping all circuit amplitudes real-valued. This simplifies the optimisation landscape compared to complex-amplitude ansätze.

**reps=3:** The layer of (Ry rotations + CNOT entanglement) is repeated 3 times. For 4 qubits, reps=3 gives approximately  $4 \times (3+1) = 16$  trainable parameters. More repetitions increase circuit depth and expressivity but also optimisation difficulty.

**Why RealAmplitudes?** It is well-suited for binary classification tasks and is a standard choice in Qiskit ML VQC examples. Real-valued amplitudes are computationally cheaper to simulate than complex amplitudes.

## 21.3 Variational Quantum Classifier (VQC)

```
# quantum/classifier.py — build_vqc()
feature_map = ZZFeatureMap(feature_dimension=4, reps=2, entanglement='full')
ansatz = RealAmplitudes(num_qubits=4, reps=3, entanglement='full')
optimizer = COBYLA(maxiter=200)
vqc = VQC(
    num_qubits=4,
    feature_map=feature_map,
    ansatz=ansatz,
    optimizer=optimizer,
)
```

**VQC training (train\_vqc):** `vqc.fit(X_train, y_train)` runs the COBYLA optimisation loop. For each iteration, COBYLA proposes parameter values  $\theta$ , the quantum circuit is evaluated for all training samples (on the simulator), the objective function (cross-entropy loss) is computed, and COBYLA updates  $\theta$ .

**VQC inference:** `predict_proba_vqc(vqc, X)` calls `vqc.predict_proba(X)` which returns  $[P(\text{class}=0), P(\text{class}=1)]$  for each sample. The positive-class probability  $P(\text{class}=1)$  is used as the seizure probability.

## 21.4 COBYLA Optimizer

**COBYLA** (Constrained Optimization By Linear Approximation) is a derivative-free optimisation algorithm. It approximates the objective function with a linear model at each iteration, using only function evaluations (not gradients).

**Why derivative-free?** Computing gradients of a quantum circuit requires the parameter-shift rule or finite differences, each requiring  $2 \times N_{\text{params}}$  additional circuit evaluations. For simulator-based training, derivative-free methods like COBYLA avoid this overhead. The tradeoff is slower convergence for high-dimensional parameter spaces.

**maxiter=200:** Training terminates after at most 200 COBYLA iterations. Each iteration requires evaluating the quantum circuit on all training samples. With ~986 training samples and 200 iterations, this is the primary driver of the ~60-minute training time.

## 22. Training Pipeline

### 22.1 Hyperparameter Search

The hyperparameter grid searched in v1.3.0:

| Hyperparameter                    | Values Tested                                 | Selected Value |
|-----------------------------------|-----------------------------------------------|----------------|
| PCA dimensions                    | [4]                                           | 4              |
| Class ratio (non-seizure:seizure) | [1.0, 2.0]                                    | 1.0            |
| Decision threshold                | [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9] | 0.6            |
| Total VQC fits                    | 40 (from evaluation_report.json)              | —              |

**Why threshold search?** The optimal probability cutoff for seizure vs non-seizure classification depends on the desired trade-off between sensitivity (fewer missed seizures) and precision (fewer false alarms). The threshold 0.60 was selected based on the inner validation F1 score: `aggregated_inner_val_f1 = 0.084` (which, while low, was the best across the searched range).

■ **Note:** Threshold 0.60 is a model decision threshold, NOT a clinical certainty score. A prediction probability of 0.62 does not mean '62% certainty of seizure'. It means the VQC's learned function outputs 0.62 for that segment, which is above the 0.60 cutoff and is therefore classified as seizure.

### 22.2 Checkpointing (checkpoint.py)

VQC training is computationally expensive (~60 minutes total). If training is interrupted (power loss, OOM, user interruption), fold-level checkpoints allow resumption without retraining completed folds.

```
# checkpoint.py – save_fold_checkpoint() – atomic write pattern
temp_dir = checkpoint_dir / f'fold_{fold_index}_tmp'
# Write all artifacts to temp directory
dill.dump(vqc, open(temp_dir / 'vqc_model.dill', 'wb'))
joblib.dump(pre_scaler, temp_dir / 'pre_scaler.joblib')
# ... write all files ...
# Atomic rename: temp → target
temp_dir.rename(target_dir)
```

**Atomic write pattern:** All checkpoint files are first written to a temporary directory (`fold_N_tmp`). Only after all files are successfully written is the directory atomically renamed to the final name (`fold_N`). This ensures that a partial checkpoint (caused by interruption during writing) is never mistakenly treated as a complete checkpoint — the target directory either doesn't exist or is fully complete.

### 22.3 Training Time and Computational Cost

| Phase                            | Duration (from evaluation_report.json) | Note                                |
|----------------------------------|----------------------------------------|-------------------------------------|
| Grid search (inner validation)   | 1.25 seconds                           | Very fast: uses cached fold results |
| Total VQC training (all 40 fits) | ~3,637 seconds (~60.6 minutes)         | On CPU quantum simulator            |
| Each VQC fit                     | ~91 seconds average                    | Derived: $3637 / 40 \approx 91s$    |

The high training time is because each `VQC.fit()` call simulates the quantum circuit on a classical CPU (Qiskit statevector/Aer simulator) for every training sample at every COBYLA iteration. This is inherently expensive: quantum simulation scales exponentially with qubit count ( $2^4 = 16$  state amplitudes for 4 qubits, manageable, but multiply by 200 iterations  $\times$  986 samples = ~197,200 circuit evaluations per fold).

## 23. Model Artifacts and Serialization

Figure 7. Model Artifact Directory Structure

```
models/
  ■■■ v1_0_0/    (archived)
  ■■■ v1_1_0/    (archived)
  ■■■ v1_2_0/    (archived)
  ■■■ v1_3_0/    ← CURRENT PRODUCTION
    ■■■ vqc_model.dill      (44 KB) ← trained VQC object
    ■■■ pre_scaler.joblib   (1 KB)  ← StandardScaler
    ■■■ pca.joblib          (2 KB)  ← PCA transformer
    ■■■ scaler.joblib       (1 KB)  ← MinMaxScaler
    ■■■ feature_config.json (609 B) ← feature schema
    ■■■ preprocessing_config.json(170 B) ← filter/segment config
    ■■■ model_metadata.json (1.7 KB)← model provenance
    ■■■ evaluation_report.json (5.5 KB)← LOS0 metrics
    ■■■ checkpoints/
      ■■■ fold_0/ ... fold_4/ ← per-fold checkpoints
      ■■■ production/        ← production checkpoint

Also present at models/ root:
  ■■■ grid_search_checkpoint.json      (91 KB)
  ■■■ grid_search_checkpoint_v1_1_0.json (28 KB)
```

Figure 7. Complete model artifact directory structure for all versions.

| Artifact                  | Format                  | Contents                                                                                     | Used During       | Why This Format                                                                               |
|---------------------------|-------------------------|----------------------------------------------------------------------------------------------|-------------------|-----------------------------------------------------------------------------------------------|
| vqc_model.dill            | dill (pickle extension) | Trained Qiskit VQC object with learned $\theta$ parameters                                   | Inference         | dill handles Qiskit's lambda functions and C extensions that standard pickle cannot serialize |
| pre_scaler.joblib         | joblib                  | StandardScaler: 20 mean values + 20 std values                                               | Inference         | joblib is optimized for NumPy arrays; faster and more memory-efficient than pickle            |
| pca.joblib                | joblib                  | PCA: components matrix (4×20), mean vector, explained variance                               | Inference         | Same as above                                                                                 |
| scaler.joblib             | joblib                  | MinMaxScaler: per-feature min/max from training PCA output                                   | Inference         | Same as above                                                                                 |
| feature_config.json       | JSON                    | Channel list, feature names, wavelet config, delta band, PCA dims, scaler range, schema hash | Inference & audit | Human-readable, version-controlled, loaded without deserialization risk                       |
| preprocessing_config.json | JSON                    | Filter bounds, segment duration, channels                                                    | Inference & audit | Human-readable subset of feature config                                                       |

|                        |      |                                                                                            |                                    |                                  |
|------------------------|------|--------------------------------------------------------------------------------------------|------------------------------------|----------------------------------|
| model_metadata.json    | JSON | Model version, VQC config, PCA EVR, training subjects, evaluation metrics, timestamp       | Inference (loads threshold), audit | Human-readable provenance record |
| evaluation_report.json | JSON | Full LOSO results: per-fold metrics, confusion matrices, grid search config, training time | Audit & reporting                  | Human-readable evaluation record |

**Schema hash:** A SHA-256 hash of the feature configuration (channels, features, wavelet settings, etc.) is stored in feature\_config.json and model\_metadata.json. At inference time, the InferenceService could verify that the current feature configuration matches the training configuration — preventing silent feature mismatch bugs if the configuration is changed after training.

■ **Important:** ARCHITECTURAL COUPLING NOTE: inference/pipeline.py imports load\_model\_artifacts() from training/save\_artifacts.py. This creates a dependency from the inference module on the training module — conceptually, inference should not need to import training code. A future refactoring should move load\_model\_artifacts() to a shared artifacts/ or core/ module, separating saving and loading responsibilities.

## 24. Inference Pipeline — inference/pipeline.py

Figure 8. Inference Pipeline

User uploads .edf file

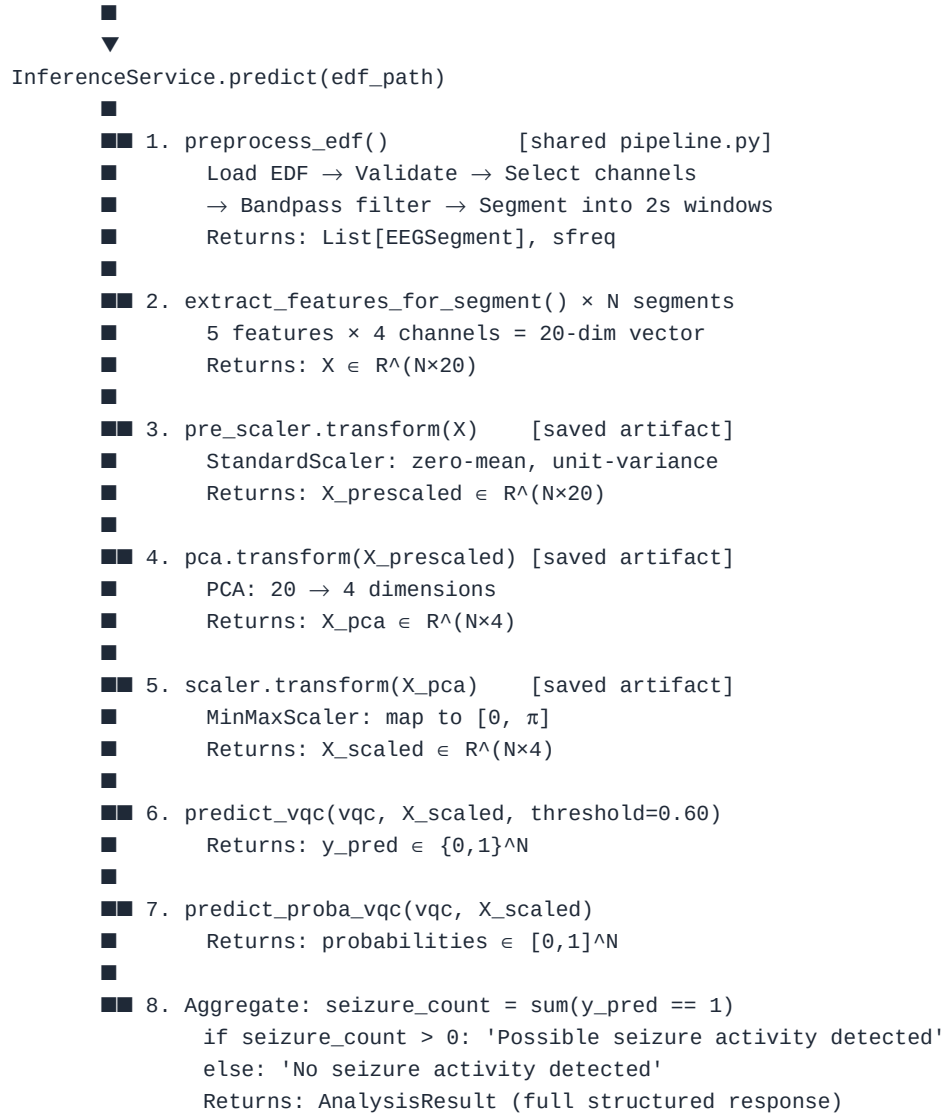


Figure 8. Complete inference pipeline from uploaded EDF to AnalysisResult.

```

# inference/pipeline.py – InferenceService.predict() – key transforms
X_prescaled = self.pre_scaler.transform(X) # NOT fit_transform()
X_pca       = self.pca.transform(X_prescaled) # NOT fit_transform()
X_scaled    = self.scaler.transform(X_pca)   # NOT fit_transform()
  
```

**transform() vs fit\_transform():** .transform() applies the already-fitted transformation. .fit\_transform() would re-fit the transformer on the new data, computing new mean/std/components from the user's EDF. This would break the training-inference mapping. Using .transform() is the correct approach.

### 24.1 Recording-Level Aggregation

```

# inference/pipeline.py – aggregation logic
seizure_count = sum(1 for sp in segment_predictions if sp.predicted_label == 1)
if seizure_count > 0:
  
```

```
prediction_summary = 'Possible seizure activity detected'
else:
    prediction_summary = 'No seizure activity detected'
```

The recording-level decision is: if ANY segment is classified as seizure, the recording is flagged. This is the most sensitive (highest recall) possible aggregation rule — a single false positive in any segment will flag the entire recording. The `recording_seizure_probability` is the mean VQC probability across all segments (regardless of threshold).

## 24.2 Decision Thresholding

**Rule:**  $P(\text{seizure}) \geq 0.60 \rightarrow \text{predicted\_label} = 1$  (Seizure);  $P(\text{seizure}) < 0.60 \rightarrow \text{predicted\_label} = 0$  (Non-Seizure).

**Effect on metrics:** A higher threshold increases precision (fewer false positives) but decreases recall (more missed seizures). A lower threshold increases recall but decreases precision. The selected threshold 0.60 was the best-performing on the inner validation set by F1-score, though F1 remained very low (0.084).

## 25. Model Evaluation

### 25.1 Evaluation Metrics

| Metric               | Formula                     | What It Measures                                                  |
|----------------------|-----------------------------|-------------------------------------------------------------------|
| Accuracy             | $TP+TN / (TP+TN+FP+FN)$     | Fraction of all predictions that are correct                      |
| Precision            | $TP / (TP+FP)$              | Of all predicted seizures, what fraction are real seizures        |
| Recall / Sensitivity | $TP / (TP+FN)$              | Of all real seizures, what fraction were detected                 |
| Specificity          | $TN / (TN+FP)$              | Of all real non-seizures, what fraction were correctly identified |
| F1-Score             | $2 \cdot P \cdot R / (P+R)$ | Harmonic mean of precision and recall                             |
| ROC-AUC              | Area under ROC curve        | NOT COMPUTED — null in evaluation_report.json                     |

TP = True Positives (seizure correctly detected), TN = True Negatives (non-seizure correctly identified), FP = False Positives (non-seizure incorrectly flagged as seizure), FN = False Negatives (seizure missed).

### 25.2 v1.3.0 Results — Fold by Fold

| Fold       | Test Subject | Segments | Accuracy       | Precision     | Recall          | F1            | Sensitivity     | Specificity    |
|------------|--------------|----------|----------------|---------------|-----------------|---------------|-----------------|----------------|
| 0          | chb01        | 11,962   | 98.05%         | 0.00%         | 0.00%           | 0.00%         | 0.00%           | 99.91%         |
| 1          | chb02        | 4,079    | 88.60%         | 7.43%         | 37.93%          | 12.43%        | 37.93%          | 89.70%         |
| 2          | chb03        | 12,600   | 94.93%         | 3.22%         | 7.39%           | 4.48%         | 7.39%           | 96.36%         |
| 3          | chb04        | 19,179   | 91.72%         | 1.32%         | 9.95%           | 2.34%         | 9.95%           | 92.54%         |
| 4          | chb05        | 9,000    | 82.64%         | 6.87%         | 36.17%          | 11.55%        | 36.17%          | 84.15%         |
| MEAN ± STD | —            | —        | 91.19% ± 5.31% | 3.77% ± 2.95% | 18.29% ± 15.67% | 6.16% ± 4.97% | 18.29% ± 15.67% | 92.53% ± 5.43% |

### 25.3 Fold-by-Fold Confusion Matrices

| Fold | Test Subject | TN     | FP    | FN  | TP  |
|------|--------------|--------|-------|-----|-----|
| 0    | chb01        | 11,729 | 10    | 223 | 0   |
| 1    | chb02        | 3,581  | 411   | 54  | 33  |
| 2    | chb03        | 11,946 | 451   | 188 | 15  |
| 3    | chb04        | 17,572 | 1,416 | 172 | 19  |
| 4    | chb05        | 7,336  | 1,382 | 180 | 102 |

### 25.4 Interpretation of Results

**High accuracy, low F1:** Accuracy of 91.19% sounds impressive but is misleading. Because seizure segments are rare (less than 5% of all segments), a model that predicts 'non-seizure' for every segment would achieve ~95% accuracy. The model's high accuracy is largely explained by correctly classifying the abundant non-seizure class, not by successfully detecting seizures.

**Fold 0 (chb01):** Precision=0%, Recall=0%, F1=0%. The model detected zero seizure segments for chb01. All 223 actual seizure segments were misclassified as non-seizure (FN=223, TP=0). This suggests the chb01 subject's EEG patterns are not well represented by training on the other four subjects.

**Subject variability:** The large standard deviations (e.g., recall: 15.67%) indicate that performance varies dramatically between subjects. chb02 and chb05 show recall ~37%, while chb01 shows 0%. This confirms the fundamental challenge of cross-patient EEG generalisation.

**False positive rate:** Across all folds, total FP =  $10+411+451+1416+1382 = 3,670$  false positives out of 56,820 segments. Many non-seizure segments are incorrectly flagged, particularly in chb04 (1,416 FP) and chb05 (1,382 FP). High false positive rates would cause alarm fatigue in clinical settings.

■ **Important:** The current model achieves a mean F1 of 6.16% and mean precision of 3.77%. This is insufficient for any clinical application. The system is a research prototype demonstrating the complete hybrid quantum-classical pipeline, not a clinically validated seizure detection tool.

## 26. Backend Architecture — api/main.py

The backend is implemented using FastAPI, an asynchronous Python web framework with automatic OpenAPI documentation generation. It exposes three REST endpoints and runs as a single-process ASGI application via Uvicorn.

```
# api/main.py – startup pattern
inference_service: InferenceService | None = None

@asynccontextmanager
async def lifespan(app: FastAPI):
    startup_event() # loads InferenceService singleton
    yield

app = FastAPI(title='Quantum EEG Signal Classifier API',
              version='1.3.0', lifespan=lifespan)
```

**Singleton pattern:** InferenceService is created once at startup and reused for all requests. This avoids re-loading the ~44 KB VQC model and sklearn artifacts on every request — important because dill.load() is relatively slow.

**CORS configuration:** The CORS middleware is configured to allow requests from http://localhost:5173 and http://127.0.0.1:5173 (Vite dev server defaults). allow\_origins=[\*] is also included for flexibility, though this is permissive and would need to be restricted in a production deployment.

### 26.1 API Endpoints

| Endpoint               | Method | Input                                              | Output                                                                                                                                                                                            | Error Cases                                                                  |
|------------------------|--------|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| GET /api/health        | GET    | None                                               | JSON: status, model_version, threshold, qubits, features, channels, segment_duration, sampling_frequency                                                                                          | 503 if Inference Service not initialized                                     |
| POST /api/analyze      | POST   | Multipart FormData: file (.edf)                    | JSON: AnalysisResult (recording_name, duration, channels_used, total_segments, seizure_segments, non_seizure_segments, prediction_summary, segment_predictions[], model_version, processing_time) | 400 if not .edf; 503 if service unavailable; 500 if inference fails          |
| POST /api/analyze-demo | POST   | None (uses test_data/chb06_01.edf or chb07_01.edf) | Same JSON structure as /api/analyze                                                                                                                                                               | 503 if service unavailable; 404 if demo file missing; 500 if inference fails |

**File upload handling (/api/analyze):** The uploaded EDF is saved to a temporary file using Python's tempfile.NamedTemporaryFile. The InferenceService processes the temp file, then os.remove() deletes it in the finally block, ensuring cleanup even if inference fails.

**Demo endpoint (/api/analyze-demo):** Checks for test\_data/chb06\_01.edf first, then test\_data/chb07\_01.edf. These are recordings from subjects NOT in the training set (chb01–chb05), making them appropriate for demonstration of inference on unseen patients.

## 27. Frontend Architecture — React + TypeScript

Figure 9. Frontend Component Architecture

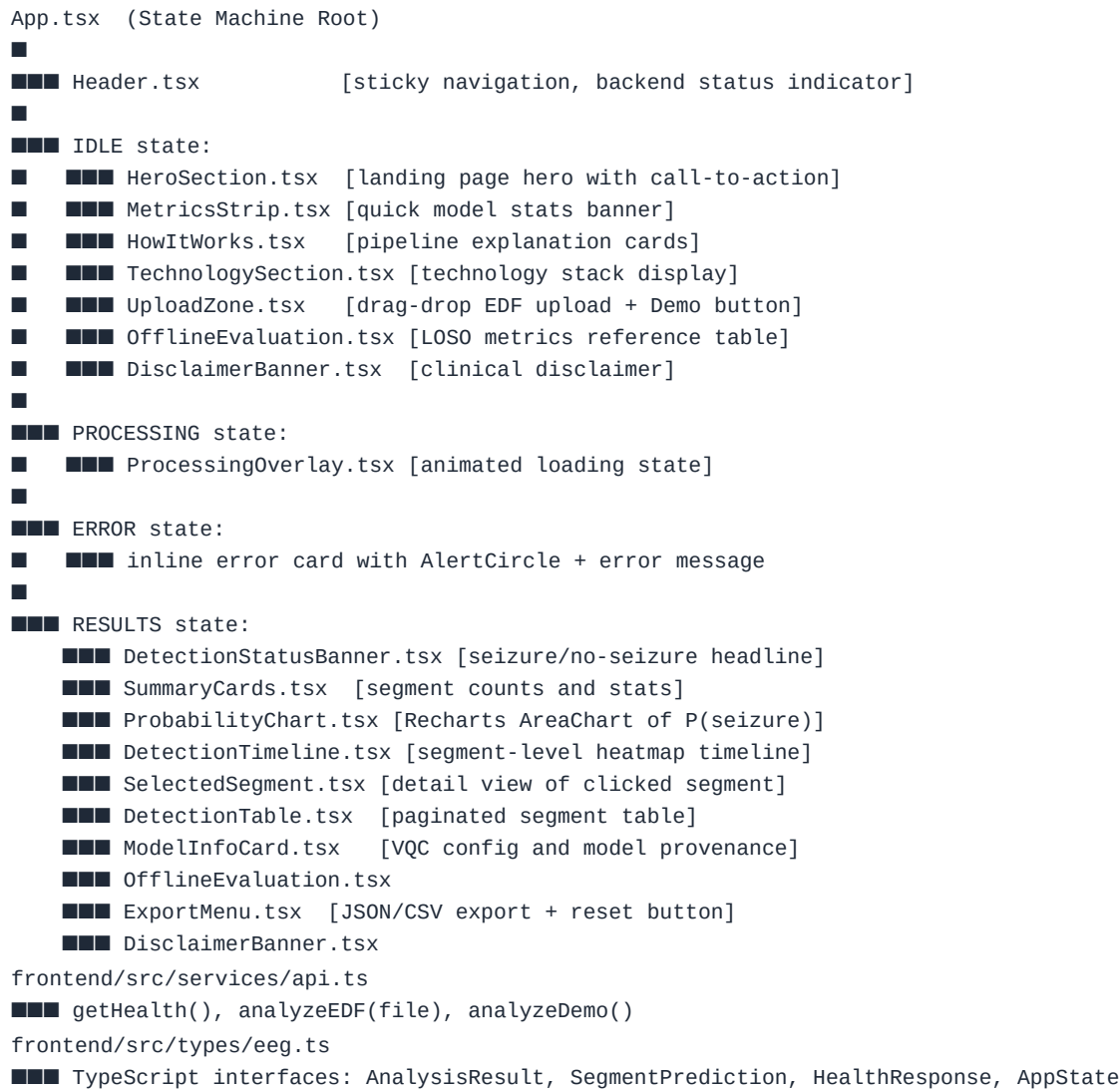


Figure 9. Frontend component tree organized by application state.

### 27.1 Application State Machine

The App.tsx component manages a single state variable `appState`: `AppState` which transitions between: `IDLE` → `PROCESSING` → `RESULTS` or `ERROR`.

```
// App.tsx – application state transitions
const [appState, setAppState] = useState<AppState>('IDLE');

const handleAnalyzeFile = async (file: File) => {
  setAppState('PROCESSING'); // show spinner
  const result = await analyzeEDF(file);
  setAnalysisResult(result);
  setAppState('RESULTS'); // show dashboard
  // on error: setAppState('ERROR')
};
```

The state machine renders entirely different UI for each state. In IDLE, the full landing page is shown. In PROCESSING, only the spinner overlay is shown. In RESULTS, the analysis dashboard is shown. In ERROR, a single error card is shown.

**Health polling:** The App polls `/api/health` every 15 seconds. If the backend is unavailable, `health=null`, and the frontend shows a warning in the Header. The UploadZone disables the upload button when `isBackendAvailable=false`.

## 27.2 Key Frontend Components

### UploadZone.tsx

Provides drag-and-drop EDF file upload. Validates file extension (.edf) on the client side before submission. Also contains the 'Run Demo Analysis' button. Sends the file to `analyzeEDF()` which calls `POST /api/analyze`.

### ProbabilityChart.tsx

Recharts AreaChart displaying seizure probability ( $P(\text{seizure})$ ) for each segment on the Y-axis, with segment index on the X-axis. A horizontal reference line is drawn at the threshold (0.60). Segments can be clicked to update the `selectedSegment` state in App.tsx.

### DetectionTimeline.tsx

Visual timeline heatmap where each 2-second segment is represented as a coloured block (red for seizure, grey for non-seizure). Allows clicking any block to inspect that segment.

### DetectionTable.tsx

Paginated data table showing all segment predictions with columns: segment index, time range (start–end seconds), label (Seizure/Non-Seizure), and seizure probability. Rows are highlighted for flagged segments.

### ModelInfoCard.tsx

Displays model provenance information fetched from `/api/health`: model version, threshold, qubit count, feature count, PCA components, channels, segment duration.

### ProcessingOverlay.tsx

Full-page spinner shown during PROCESSING state. Includes animated elements and informational steps showing the inference stages (Loading EDF, Extracting Features, Running VQC, etc.)

### ExportMenu.tsx

Provides download buttons for exporting the analysis result as JSON or CSV. Also contains the 'New Analysis' reset button.

### DisclaimerBanner.tsx

Static disclaimer stating the system is not validated for clinical use. Always displayed to ensure users understand the research-prototype nature.

### OfflineEvaluation.tsx

Static component displaying the LOSO evaluation results table, reminding users of the model's actual performance characteristics.

### **services/api.ts**

Centralised API client with three functions: `getHealth()`, `analyzeEDF(file)`, `analyzeDemo()`. Uses the Fetch API with `FormData` for file upload. Throws `ApiError` with HTTP status code for error handling in `App.tsx`.

## 28. End-to-End User Workflow

### Figure 10. End-to-End User Workflow

Step 1: User opens application (`http://localhost:5173`)  
 App.tsx mounts → fetches GET `/api/health` → displays Header status indicator  
 IDLE state: renders HeroSection, UploadZone, OfflineEvaluation  
 ■

Step 2: User drags/drops EDF file into UploadZone  
 UploadZone validates file extension (`.edf`)  
`handleAnalyzeFile()` called → `setAppState('PROCESSING')`  
 ■

Step 3: Frontend sends POST `/api/analyze` with `FormData { file: chb06_01.edf }`  
`ProcessingOverlay` renders (animated loading UI)  
 ■

Step 4: FastAPI receives file → saves to temp file → calls `InferenceService.predict()`  
 EDF Loading → Validation → Channel Selection → Filtering → Segmentation  
 Feature Extraction × N segments → `pre_scaler.transform` → `pca.transform`  
 → `scaler.transform` → `predict_vqc` → `predict_proba_vqc`  
 AnalysisResult assembled → temp file deleted → JSON response returned  
 ■

Step 5: Frontend receives JSON → `setAnalysisResult` → `setAppState('RESULTS')`  
 Results dashboard renders:  
 - `DetectionStatusBanner`: 'Possible seizure activity detected' / 'No seizure'  
 - `SummaryCards`: total segments, seizure count, non-seizure count  
 - `ProbabilityChart`: probability for each segment (with 0.60 threshold line)  
 - `DetectionTimeline`: colour-coded segment blocks  
 - `DetectionTable`: searchable, paginated segment table  
 - `ModelInfoCard`: VQC config, model version, threshold  
 - `OfflineEvaluation`: LOSO metrics reference  
 ■

Step 6: User clicks a segment in the chart/timeline/table  
`setSelectedSegment(segment)` → `SelectedSegment.tsx` re-renders with details  
 ■

Step 7: User clicks `ExportMenu` → downloads `results.json` or `results.csv`  
 OR clicks 'New Analysis' → `handleReset()` → `setAppState('IDLE')`

*Figure 10. Complete user workflow from browser open to results view.*

## 29. Code Walkthrough

This section provides a detailed walkthrough of the most important code blocks, explaining what each block does, how it works, and why it is written this way.

### 29.1 eeg\_processing/loader.py

**Purpose:** Load an EDF file using MNE without loading data into memory. Provides lazy access to the EDF header (channel names, sampling rate, duration) without the memory overhead of loading potentially hundreds of MB of signal data.

```
def load_edf(file_path: str | Path, preload: bool = False) -> mne.io.Raw:
    file_path = Path(file_path)
    if not file_path.exists():
        raise EdfFileNotFoundError(f'EDF file not found: {file_path}')
    if file_path.suffix.lower() != '.edf':
        raise EdfInvalidFormatError(...)
    raw = mne.io.read_raw_edf(str(file_path), preload=preload, verbose=False)
    return raw
```

Line 1: Type hint uses Union (str | Path) so callers can pass either string paths or pathlib.Path objects. Line 2: Converts to Path for consistent behaviour. Lines 3-5: Validate existence before passing to MNE (MNE's error message would be less clear). Lines 6-8: Load with MNE; preload=False means only header is read.

### 29.2 eeg\_processing/feature\_extraction.py — Core Extract Function

```
def extract_features_for_segment(segment_data, sfreq, config) -> np.ndarray:
    features: list[float] = []
    for ch_idx in range(config.n_channels): # 4 channels
        ch_signal = segment_data[ch_idx] # shape: (512,) at 256 Hz
        features.append(_wavelet_approx_energy(ch_signal, 'db4', 4))
        features.append(_wavelet_energy(ch_signal, 'db4', 4))
        features.append(_shannon_entropy(ch_signal))
        features.append(_delta_band_power(ch_signal, sfreq, 0.5, 4.0))
        features.append(_hjorth_mobility(ch_signal))
    feature_vec = np.array(features, dtype=np.float64) # shape: (20,)
    bad_mask = ~np.isfinite(feature_vec)
    if bad_mask.any():
        feature_vec[bad_mask] = 0.0 # replace NaN/Inf with 0
    return feature_vec
```

The outer loop iterates over 4 channels. For each channel, 5 features are appended to the list in deterministic order. The resulting list has 4×5=20 elements, always in the same order. The bad\_mask check replaces any NaN or infinite values with 0.0 — this handles edge cases where very short or silent segments produce undefined feature values (e.g., 0/0 in delta power).

### 29.3 quantum/classifier.py — build\_vqc()

```
def build_vqc(config: VQCConfig, num_qubits: int = 4) -> VQC:
    feature_map = ZZFeatureMap(
        feature_dimension=num_qubits, # 4
        reps=config.feature_map_reps, # 2
        entanglement=config.entanglement, # 'full'
```

```

    )
    ansatz = RealAmplitudes(
        num_qubits=num_qubits,      # 4
        reps=config.ansatz_reps,    # 3
        entanglement=config.entanglement,
    )
    optimizer = COBYLA(maxiter=config.maxiter) # 200
    return VQC(num_qubits=4, feature_map=feature_map,
               ansatz=ansatz, optimizer=optimizer)

```

This function is a factory that constructs a completely untrained VQC. The `num_qubits` parameter matches the PCA output dimension (4), ensuring the feature map expects exactly 4 input values. The function is called at the start of each LOSO fold to create a fresh VQC before training.

## 29.4 quantum/classifier.py — `predict_proba_vqc()`

```

def predict_proba_vqc(vqc: VQC, X: np.ndarray) -> np.ndarray:
    proba = vqc.predict_proba(X)
    if proba.ndim == 2 and proba.shape[1] >= 2:
        return proba[:, 1] # positive class probability
    return proba.ravel()

```

`VQC.predict_proba()` output shape varies between Qiskit versions. For binary classification it typically returns (N, 2) with columns [P(class=0), P(class=1)]. The code handles both this case (`ndim==2, shape[1]>=2`: return column 1) and the alternative case where a 1D array is returned. This defensive handling prevents runtime errors from Qiskit version differences.

## 29.5 inference/pipeline.py — `InferenceService.__init__()`

```

def __init__(self, artifacts_dir: Path, model_version: str, ...):
    self.artifacts = load_model_artifacts(artifacts_dir, model_version)
    self.vqc = self.artifacts['vqc']
    self.pre_scaler = self.artifacts['pre_scaler']
    self.pca = self.artifacts['pca']
    self.scaler = self.artifacts['scaler']
    self.metadata = self.artifacts['metadata']
    self.threshold = self.metadata.get('threshold', 0.5)

```

The `InferenceService` loads all artifacts once during initialisation and stores them as instance attributes. All subsequent calls to `predict()` reuse the same objects without re-loading from disk. The threshold is read from `model_metadata.json` (0.60 for v1.3.0) and used for every prediction.

## 29.6 api/main.py — `/api/analyze` endpoint

```

@app.post('/api/analyze')
async def analyze_edf(file: UploadFile = File(...)):
    if not filename.lower().endswith('.edf'):
        raise HTTPException(400, 'Only .edf files are supported')
    with tempfile.NamedTemporaryFile(delete=False, suffix='.edf') as f:
        shutil.copyfileobj(file.file, f)
        temp_path = Path(f.name)
    try:
        result = inference_service.predict(temp_path)
        return asdict(result)

```

```
finally:
```

```
    os.remove(temp_path) # always cleanup
```

The try/finally block ensures the temporary file is deleted even if inference raises an exception. `asdict()` converts the `AnalysisResult` dataclass to a plain dictionary, which FastAPI serialises as JSON. The filename check prevents non-EDF files from reaching the inference pipeline.

## 30. Engineering Design Decisions — "Why?"

---

### Q: Why MNE instead of manually parsing EDF?

EDF has a complex binary specification with multiple variants (EDF, EDF+, BDF, EDF+D). MNE provides a battle-tested implementation covering all EDF variants, handles encoding issues, supports lazy loading, and integrates seamlessly with channel selection and filtering. Writing a custom EDF parser would require significant effort and testing.

### Q: Why 4 specific EEG channels?

Four channels cover multiple brain regions (frontal, central, parieto-occipital), providing spatial coverage while keeping feature dimensionality manageable. The selection was a design decision, not an experimentally optimised choice. Using all 23 channels would produce a 115-dimensional (23×5) feature vector requiring PCA to ~4 components, potentially discarding more information.

### Q: Why 0.5–40 Hz bandpass?

This range covers all clinically relevant EEG bands (delta, theta, alpha, beta) while attenuating DC drift (<0.5 Hz), baseline wander, and high-frequency muscle artifact (>40 Hz). It is a standard clinical EEG filter range used in most seizure detection literature.

### Q: Why 2-second non-overlapping segments?

Two seconds captures at least one full cycle of the lowest band of interest (0.5 Hz). Non-overlapping prevents correlated training samples. The 2-second duration balances spectral resolution against temporal resolution and is commonly used in EEG classification literature.

### Q: Why 5 engineered features instead of raw EEG?

Raw EEG at 256 Hz produces 512 samples per 2-second segment per channel. The VQC requires the number of features to equal the number of qubits. Providing 2048 raw values would require 2048 qubits — far beyond simulation capability. Feature engineering compresses the signal to interpretable representations.

### Q: Why StandardScaler before PCA?

PCA is scale-sensitive. Features measured in different units (energy in  $\text{mV}^2$ , entropy in bits, power ratio unitless) have wildly different magnitudes. StandardScaler normalises all 20 features to zero mean / unit variance before PCA, ensuring each feature contributes proportionally to the principal components.

### Q: Why PCA 20→4?

The VQC has 4 qubits. Each qubit corresponds to one quantum angle, which is one feature. Therefore exactly 4 features are required. PCA provides the mathematically optimal linear compression (maximum variance retained).

### Q: Why MinMaxScaler to $[0, \pi]$ ?

ZZFeatureMap encodes features as quantum rotation angles. Rotation gates act on  $[0, 2\pi]$ . Using  $[0, \pi]$  covers the meaningful half of the rotation space. Values outside  $[0, 2\pi]$  would alias due to trigonometric periodicity.

### Q: Why LOSO cross-validation?

EEG is highly patient-specific. Random segment splits would allow correlated segments from the same patient in both train and test — inflating performance. LOSO ensures test subjects are completely unseen during training, measuring true cross-patient generalisation.

### Q: Why class ratio 1:1 balancing?

The inner validation selected ratio 1.0 as producing the best inner F1. Balancing prevents the VQC from converging to a trivial all-non-seizure solution caused by the minority-class problem in heavily imbalanced data.

### Q: Why dill for VQC, joblib for sklearn objects?

Qiskit VQC objects contain lambda functions, Cython extensions, and custom C objects that standard pickle cannot handle. dill extends pickle to serialise arbitrary Python objects including closures. sklearn objects (StandardScaler, PCA, MinMaxScaler) contain only NumPy arrays and are handled by joblib — which uses memory-mapped files for efficient large-array serialisation.

### Q: Why FastAPI instead of Flask?

FastAPI is built on ASGI and supports async I/O natively, important when handling file uploads. It provides automatic OpenAPI documentation, Pydantic-based request validation, and type-annotated handler functions. Flask requires more boilerplate for the same functionality.

### Q: Why React with TypeScript?

TypeScript provides compile-time type checking for the AnalysisResult and SegmentPrediction JSON response shapes. If the backend API changes, TypeScript errors immediately identify affected frontend components. React's component model enables each visualisation (chart, timeline, table) to be developed and tested independently.

### Q: Why Recharts?

Recharts is built on top of D3 and designed specifically for React. It provides native React components (AreaChart, LineChart, etc.) with responsive containers. The integration with React state (selectedSegmentIndex) is natural with Recharts' callback props (onMouseDown, onClick events).

### 31. Mathematical Formulations

| Formula                                                      | Description                                                                             |
|--------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| $X \in \mathbb{R}^{(N \times 20)}$                           | Feature matrix: N segments, 20 features each                                            |
| $y \in \{0,1\}^N$                                            | Label vector: 0=non-seizure, 1=seizure                                                  |
| $z = (x - \mu) / \sigma$                                     | StandardScaler standardization ( $\mu$ =training mean, $\sigma$ =training std)          |
| $X_{pca} = X_{std} \cdot W^T$                                | PCA projection ( $W \in \mathbb{R}^{(4 \times 20)}$ : top-4 eigenvectors of covariance) |
| $x_{scaled} = (x - x_{min}) / (x_{max} - x_{min}) \cdot \pi$ | MinMax scaling to $[0, \pi]$                                                            |
| $ \psi\rangle = U_{\phi}(x) 0\rangle^{\otimes 4}$            | Quantum state after ZZFeatureMap encoding ( $U_{\phi}$ parameterized by $x$ )           |
| $ \psi(\theta)\rangle = U_W(\theta) \psi\rangle$             | Quantum state after RealAmplitudes ansatz ( $\theta$ : trainable parameters)            |
| $p =  \langle 1 \psi(\theta)\rangle ^2$                      | Positive-class probability from quantum measurement                                     |
| $\hat{y} = 1 \text{ if } p \geq 0.60, \text{ else } 0$       | Binary decision rule with threshold 0.60                                                |
| $\text{Precision} = TP / (TP + FP)$                          | Fraction of seizure predictions that are correct                                        |
| $\text{Recall} = TP / (TP + FN)$                             | Fraction of actual seizures that are detected                                           |
| $F1 = 2 \cdot P \cdot R / (P + R)$                           | Harmonic mean of precision and recall                                                   |
| $\text{Sensitivity} = TP / (TP + FN)$                        | Identical to recall; probability of detecting a seizure                                 |
| $\text{Specificity} = TN / (TN + FP)$                        | Probability of correctly identifying non-seizure                                        |
| $H(X) = -\sum p(x) \log p(x)$                                | Shannon entropy over amplitude distribution                                             |
| $\text{Mobility} = \sigma(dx/dt) / \sigma(x)$                | Hjorth Mobility; $\sigma$ denotes standard deviation                                    |

## 32. Version History and Evolution

Four model version directories are present: v1\_0\_0, v1\_1\_0, v1\_2\_0, v1\_3\_0. The grid\_search\_checkpoint\_v1\_1\_0.json (28 KB) and grid\_search\_checkpoint.json (91 KB) at the models/ root record the evolution of the hyperparameter search.

| Version | Key Changes                                                                                                                                         | Feature Dimensions                          | Evaluation Method                       |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|-----------------------------------------|
| v1.0.0  | Initial implementation (described in IMPLEMENTATION_PLAN.md, app.yaml references this version)                                                      | Earlier implementation — likely 16 features | Basic evaluation                        |
| v1.1.0  | Grid search checkpoint v1.1.0 present — suggests hyperparameter search was added                                                                    | Transitional                                | LOSO with grid search                   |
| v1.2.0  | Version directory present; specific changes not documented in accessible logs                                                                       | Transitional                                | Nested LOSO                             |
| v1.3.0  | Current production: 20 features (4×5), nested LOSO, class_ratio search [1.0, 2.0], threshold search 0.1–0.9, COBYLA maxiter=200, fold checkpointing | 20 (4 channels × 5 features)                | Nested LOSO (rotating inner validation) |

### Evolution of Feature Representation

Earlier documentation (IMPLEMENTATION\_PLAN.md, docstring in extract\_features\_for\_segment) references '16-dimensional' features. The current implementation (v1.3.0) uses 20 features (4 channels × 5 features). The transition from 16 to 20 features was made by adding one feature (likely wavelet\_approx\_energy or hjorth\_mobility was added to an earlier 4-feature-per-channel set). The code is correct at 20 features; only the docstring was not updated.

### Known Stale Configuration

■ **Important:** config/app.yaml references 'model: version: v1.0.0'. The actual api/main.py hardcodes MODEL\_VERSION = 'v1\_3\_0'. The app.yaml was not updated when the model was upgraded. The FastAPI backend ignores app.yaml entirely — it uses the hardcoded version string. This is a configuration drift issue.

## 33. Failed Experiments and Lessons Learned

---

### 33.1 Class Ratio 2.0

Hypothesis: Using 2:1 non-seizure:seizure ratio during training would help the model better handle the real-world imbalance and reduce false positives.

Result: The inner validation selected ratio 1.0 over 2.0 for all 5 folds (`selected_configuration.class_ratio = 1.0`). Ratio 2.0 produced lower F1 in inner validation, suggesting the additional non-seizure samples confused the VQC rather than improving specificity.

### 33.2 Threshold Values Outside 0.60

All threshold values from 0.1 to 0.9 were tested in the grid search. The inner validation F1 was maximised at `threshold=0.60` (`aggregated_inner_val_f1 = 0.084`). However, even the best-performing threshold produces an extremely low F1, indicating that the problem is not threshold selection but fundamental model performance.

Lower thresholds (0.1–0.3) would increase recall but at the cost of extremely high false positive rates. Higher thresholds (0.7–0.9) reduce false positives but cause even more missed seizures. Threshold 0.60 was selected as the best available compromise, not as an optimal solution.

### 33.3 Diagnostic Testing on Unseen Subjects

Several diagnostic scripts are present in the repository root: `diagnostic.py`, `diagnostic_candidates.py`, `diagnostic_postprocess.py`, `diagnostic_temporal.py`, `diagnostic_timing.py`, `diagnostic_tp_fn.py`. These were used to test the model on recordings from chb06 and chb07 (subjects NOT in the training set).

The demo endpoint (`/api/analyze-demo`) is configured to use `test_data/chb06_01.edf` or `test_data/chb07_01.edf`. These recordings serve as out-of-distribution test cases for demonstration purposes — they are not part of the LOSO evaluation and should not be interpreted as formal validation results.

### 33.4 Temporal Post-Processing Experiments

`diagnostic_temporal.py` suggests experiments with temporal smoothing/post-processing (e.g., 'N-of-M' rules: require N consecutive or nearby positive segments to count as a seizure). This approach is common in clinical EEG systems to reduce false positives caused by isolated noisy segments. The current production pipeline does NOT implement temporal post-processing — the recording-level decision uses the simplest possible rule (any positive segment → record flagged). The temporal experiments were not adopted in v1.3.0.

## 34. Current Limitations

| Limitation                        | Status    | Description                                                                                                                                                                                                                                          |
|-----------------------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Very low precision and F1         | Confirmed | Mean F1=6.16%, precision=3.77%. The model cannot reliably detect seizures. Most positive predictions are false positives.                                                                                                                            |
| Fold 0 (chb01) complete failure   | Confirmed | Zero seizure segments detected for chb01 (TP=0, recall=0%). Subject-to-subject variability exceeds the model's generalisation ability.                                                                                                               |
| Limited training subjects         | Confirmed | Only 5 subjects from CHB-MIT used. Insufficient diversity to learn general seizure patterns.                                                                                                                                                         |
| No ROC-AUC                        | Confirmed | ROC-AUC = null in evaluation_report.json. roc_auc is explicitly set to None in evaluate.py.                                                                                                                                                          |
| No classical baseline comparison  | Confirmed | No SVM, logistic regression, or random forest baseline was implemented. Cannot determine if VQC outperforms classical alternatives.                                                                                                                  |
| Docstring inconsistency           | Confirmed | extract_features_for_segment() docstring says '16-dimensional' but code produces 20 dimensions.                                                                                                                                                      |
| Architectural coupling            | Confirmed | inference/pipeline.py imports load_model_artifacts from training/save_artifacts.py — inference depends on training code.                                                                                                                             |
| Stale app.yaml                    | Confirmed | app.yaml references v1.0.0 but api/main.py uses v1_3_0.                                                                                                                                                                                              |
| Docker not implemented            | Confirmed | Planned in IMPLEMENTATION_PLAN.md but no Dockerfile/docker-compose.yml found.                                                                                                                                                                        |
| Simulator-only quantum execution  | Confirmed | All VQC training and inference runs on Qiskit's classical simulator — no real quantum hardware used.                                                                                                                                                 |
| No artifact contamination removal | Confirmed | Bandpass filtering attenuates some artifacts but no ICA, artifact rejection, or reference re-montaging is applied.                                                                                                                                   |
| Serialization compatibility risk  | Potential | dill-serialized Qiskit VQC objects may not be loadable if Qiskit version changes significantly. The load_model_artifacts() function includes a QiskitVQCUnpickler fallback for Qiskit 2.5 compatibility, indicating this issue has already occurred. |
| No external validation            | Confirmed | Model evaluated only on CHB-MIT. Performance on other EEG datasets or recording systems is unknown.                                                                                                                                                  |

## 35. Future Improvements

---

- **More training subjects:** Include all 23 CHB-MIT subjects and other public EEG datasets to improve generalisation.
- **Classical baseline comparison:** Implement SVM, logistic regression, and random forest classifiers with the same feature set to quantify the contribution (if any) of the quantum component.
- **Corrected wavelet feature labeling:** Update docstrings and verify wavelet coefficient assignments with literature references.
- **Additional EEG channels:** Experiment with more channels (8, 16, 23) to determine if spatial coverage improves classification.
- **Improved class imbalance strategies:** Test SMOTE (Synthetic Minority Oversampling), cost-sensitive learning, and focal loss instead of simple undersampling.
- **Temporal post-processing:** Implement N-of-M majority voting or sliding window smoothing to reduce false positives.
- **PR-AUC metric:** Compute Precision-Recall AUC as a class-imbalance-aware alternative to accuracy/ROC-AUC.
- **Alternative quantum ansatz:** Test EfficientSU2 or HardwareEfficient ansatzes with more expressive parameter sets.
- **Higher qubit count:** Explore 6–8 qubits with corresponding PCA dimensions to increase circuit expressivity.
- **Alternative optimizer:** Test SPSA (Simultaneous Perturbation Stochastic Approximation) or gradient-based optimizers.
- **Model calibration:** Apply Platt scaling or isotonic regression to calibrate VQC probability outputs.
- **Docker containerisation:** Implement Dockerfile and docker-compose.yml for one-command deployment.
- **Refactor artifact loading:** Move load\_model\_artifacts() to a shared core/artifacts.py module, decoupling inference from training code.
- **Update app.yaml:** Fix the stale model version reference from v1.0.0 to v1.3.0.
- **Security improvements:** Add file size limits, authentication, and restrict CORS origins before any public deployment.
- **Streaming inference:** For very long recordings, process segments in streaming batches rather than loading all predictions into memory.

## 36. Implementation Status Table

| Feature                                       | Planned | Implemented | Tested | Notes                              |
|-----------------------------------------------|---------|-------------|--------|------------------------------------|
| EDF upload & loading                          | ✓       | ✓           | ✓      | MNE-based, lazy loading            |
| EDF validation (channels, sfreq, duration)    | ✓       | ✓           | ✓      | validator.py                       |
| 4-channel selection                           | ✓       | ✓           | ✓      | FP1-F7, F3-C3, FZ-CZ, P4-O2        |
| Bandpass filter 0.5–40 Hz                     | ✓       | ✓           | ✓      | FIR filter via MNE                 |
| 2-second non-overlapping segmentation         | ✓       | ✓           | ✓      | segmentation.py                    |
| CHB-MIT annotation parsing                    | ✓       | ✓           | ✓      | annotations.py (regex)             |
| Segment labeling (overlap rule)               | ✓       | ✓           | ✓      | build_dataset.py                   |
| 20-dim feature extraction                     | ✓       | ✓           | ✓      | feature_extraction.py              |
| StandardScaler (pre-PCA)                      | ✓       | ✓           | ✓      | prepare_dataset.py                 |
| PCA 20→4                                      | ✓       | ✓           | ✓      | 72.46% variance retained           |
| MinMaxScaler to [0, $\pi$ ]                   | ✓       | ✓           | ✓      | prepare_dataset.py                 |
| VQC (4 qubits, ZZFeatureMap, RealAmplitudes)  | ✓       | ✓           | ✓      | quantum/classifier.py              |
| COBYLA optimizer (maxiter=200)                | ✓       | ✓           | ✓      | quantum/classifier.py              |
| LOSO cross-validation                         | ✓       | ✓           | ✓      | prepare_dataset.py                 |
| Nested LOSO hyperparameter search             | ✓       | ✓           | ~      | Inner validation F1 very low       |
| Class ratio balancing (1:1)                   | ✓       | ✓           | ✓      | training-only                      |
| Threshold search (0.1–0.9)                    | ✓       | ✓           | ✓      | Selected 0.60                      |
| Fold-level checkpointing (atomic)             | ✓       | ✓           | ✓      | checkpoint.py                      |
| Artifact serialization (dill + joblib + JSON) | ✓       | ✓           | ✓      | save_artifacts.py                  |
| Feature schema hash                           | ✓       | ✓           | ~      | SHA-256; not verified at inference |
| InferenceService (stateless)                  | ✓       | ✓           | ✓      | inference/pipeline.py              |
| FastAPI backend                               | ✓       | ✓           | ✓      | api/main.py                        |
| /api/health endpoint                          | ✓       | ✓           | ✓      | Returns model metadata             |
| /api/analyze endpoint                         | ✓       | ✓           | ✓      | File upload + inference            |
| /api/analyze-demo endpoint                    | ✓       | ✓           | ~      | Requires chb06/chb07 in test_data  |
| React frontend (Vite + TypeScript)            | ✓       | ✓           | ✓      | 19 components                      |
| ProbabilityChart (Recharts)                   | ✓       | ✓           | ✓      | AreaChart with threshold line      |
| DetectionTimeline                             | ✓       | ✓           | ✓      | Colour-coded segment blocks        |
| DetectionTable (paginated)                    | ✓       | ✓           | ✓      | All segment predictions            |
| ExportMenu (JSON/CSV)                         | ✓       | ~           | ~      | ExportMenu.tsx present             |
| DisclaimerBanner                              | ✓       | ✓           | ✓      | Always displayed                   |
| Docker deployment                             | ✓       | ✗           | ✗      | No Dockerfile found                |
| Clinical validation                           | —       | ✗           | ✗      | Not in scope; not intended         |

Legend: ✓ Implemented and verified ~ Implemented but not fully verified ✗ Not implemented

## 37. Final Technical Assessment

---

### What Has Been Successfully Implemented

The Quantum EEG Signal Classifier successfully implements a complete, end-to-end hybrid classical-quantum machine learning pipeline for EEG seizure detection. This includes: (1) a robust EEG processing pipeline handling EDF loading, multi-stage validation, channel selection, FIR bandpass filtering, non-overlapping segmentation, annotation-based labeling, and 20-dimensional feature extraction; (2) a principled training workflow using nested LOSO cross-validation with inner-loop hyperparameter selection; (3) a complete quantum classification system using Qiskit's VQC with ZZFeatureMap and RealAmplitudes ansatz trained via COBYLA; (4) proper model serialization for deployment-ready inference; (5) a FastAPI REST API backend with file upload handling; and (6) a polished React frontend with probability charts, detection timeline, paginated results table, and model information display.

### What Has Been Experimentally Demonstrated

The system successfully processes EDF files, extracts features, runs VQC inference, and displays results in the browser. Training completed for 5 subjects with 40 VQC fits in approximately 60 minutes. The system correctly applies saved preprocessing transforms during inference without re-fitting (preventing data leakage). Fold-level checkpointing enables interrupted training to be resumed.

### Current Model Performance

The v1.3.0 model achieves: mean accuracy 91.19% (misleading due to class imbalance), mean precision 3.77%, mean recall 18.29%, mean F1-score 6.16%, mean specificity 92.53%. The model completely fails on one of five test subjects (chb01: F1=0%, TP=0). Subject-to-subject variability is high. The model generates substantial false positives (3,670 across all test folds).

### Engineering Limitations

Key engineering limitations: (1) inference/pipeline.py is coupled to training/save\_artifacts.py; (2) app.yaml references an outdated model version; (3) a docstring incorrectly states '16-dimensional' features; (4) no Docker deployment; (5) VQC serialization requires a custom unpickler for Qiskit 2.5 compatibility; (6) CORS is configured with allow\_origins=[\*].

### Clinical Limitations

The system has NOT been clinically validated. It was not developed following medical device regulatory pathways. The training dataset (5 paediatric patients, CHB-MIT) is insufficient for clinical validation. Results have not been compared against clinical EEG reading by qualified neurologists. The system must not be used to inform clinical decisions about individual patients.

### Most Important Future Improvements

Priority improvements: (1) increase training subjects across multiple EEG datasets; (2) implement a classical baseline comparison; (3) add temporal post-processing to reduce false positives; (4) compute PR-AUC as the primary evaluation metric; (5) address the architectural coupling between inference and training modules; (6) implement Docker deployment.

### Quantum vs Classical

This project uses quantum machine learning (VQC on a classical simulator). No quantum advantage has been demonstrated or claimed. The VQC is significantly slower to train than classical alternatives (~60 minutes vs potentially seconds for SVM/logistic regression on the same data). The primary value of the quantum component is educational and exploratory — demonstrating that a complete hybrid classical–quantum pipeline can be built end-to-end on a real biomedical dataset.

## 38. Reproducibility Guide

---

### A. Environment Setup

```
# Create virtual environment
python -m venv .venv
source .venv/bin/activate # Linux/Mac
.venv\Scripts\activate    # Windows
# Install Python dependencies
pip install -e '.[dev]'
# Or: pip install -r requirements.txt
```

### B. Training (requires CHB-MIT dataset)

```
# Place CHB-MIT data in data/training/
# Structure: data/training/chb01/chb01_XX.edf + chb01-summary.txt
# Run training script (from project root)
python training/train_vqc.py
# OR use the Antigravity Training Pipeline Prompt.md workflow
```

### C. Backend Startup

```
# Ensure models/v1_3_0/ contains all artifacts
# Start FastAPI server
uvicorn api.main:app --reload --port 8000
# Backend available at http://localhost:8000
# API docs at http://localhost:8000/docs
```

### D. Frontend Startup

```
cd frontend
npm install
npm run dev
# Frontend available at http://localhost:5173
```

### E. Inference (End User)

The end user does NOT need the CHB-MIT training dataset. All required artifacts (VQC model, PCA, scalars, configuration) are pre-computed and stored in `models/v1_3_0/`. The user only needs to: (1) start the backend, (2) start the frontend, (3) upload any compatible `.edf` recording (with required channels).

## 39. Glossary

**Ansatz:** The parameterised quantum circuit that serves as the trainable component of a VQC. The RealAmplitudes ansatz uses Ry rotations and CNOT gates.

**CHB-MIT:** Boston Children's Hospital – MIT scalp EEG database. A public dataset of paediatric EEG recordings including seizure events.

**COBYLA:** Constrained Optimization By Linear Approximation. A derivative-free numerical optimisation algorithm suitable for quantum circuit parameter optimisation.

**EDF:** European Data Format. A standard binary file format for continuous multichannel physiological signals, including EEG.

**EEG:** Electroencephalography. A non-invasive technique recording the brain's electrical activity via scalp electrodes.

**F1-Score:** Harmonic mean of precision and recall. A balanced metric for classification with class imbalance.

**False Negative (FN):** An actual seizure segment that the model predicts as non-seizure (missed seizure).

**False Positive (FP):** A non-seizure segment that the model predicts as seizure (false alarm).

**Feature Map:** A quantum circuit that encodes classical data into quantum states. ZZFeatureMap creates entangled states from input features.

**Hjorth Mobility:** A time-domain EEG complexity measure:  $\text{std}(\text{derivative}) / \text{std}(\text{signal})$ . Related to mean frequency.

**LOSO:** Leave-One-Subject-Out. A cross-validation strategy where one complete subject is held out as the test set at each fold.

**MinMaxScaler:** Scales features to a specified range [min, max] using:  $x_{\text{scaled}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}}) * (\text{max} - \text{min}) + \text{min}$ .

**PCA:** Principal Component Analysis. Linear dimensionality reduction that maximises retained variance. Used to reduce 20 features to 4.

**Precision:**  $TP / (TP + FP)$ . Fraction of predicted seizures that are actual seizures.

**Qubit:** Quantum bit. The fundamental unit of quantum information, capable of existing in superposition of states  $|0\rangle$  and  $|1\rangle$ .

**QML:** Quantum Machine Learning. The application of quantum computing to machine learning problems.

**Qiskit:** IBM's open-source quantum computing software development kit, providing quantum circuit building blocks and simulators.

**RealAmplitudes:** A Qiskit quantum circuit ansatz using only real-valued amplitude rotations (Ry gates) and CNOT entanglement.

**Recall / Sensitivity:**  $TP / (TP + FN)$ . Fraction of actual seizures that were detected.

**ROC-AUC:** Area Under the Receiver Operating Characteristic Curve. Not computed in v1.3.0 (null in evaluation report).

**Segmentation:** Division of a continuous EEG recording into fixed-duration non-overlapping windows (2 seconds in this project).

**Shannon Entropy:** An information-theoretic measure of signal complexity:  $H = -\sum p(x) \log_2 p(x)$ . High entropy = more irregular signal.

**Specificity:**  $TN / (TN + FP)$ . Fraction of non-seizure segments correctly identified.

**StandardScaler:** Normalises features to zero mean and unit variance:  $z = (x - \mu) / \sigma$ . Fitted on training data only.

**Threshold:** Decision cutoff for converting probabilities to binary predictions. At 0.60:  $P(\text{seizure}) \geq 0.60 \rightarrow \text{predicted seizure}$ .

**True Negative (TN):** A non-seizure segment correctly identified as non-seizure.

**True Positive (TP):** A seizure segment correctly identified as seizure.

**VQC:** Variational Quantum Classifier. A hybrid classical-quantum algorithm using a parameterised quantum circuit trained by a classical optimiser.

**ZZFeatureMap:** A Qiskit feature map that encodes classical features using single-qubit rotations and two-qubit ZZ interactions.

# Appendix A — Project Directory Tree

|                                |                      |                                         |
|--------------------------------|----------------------|-----------------------------------------|
| Quantum-EEG-Signal-Classifier/ |                      |                                         |
| ■■■ api/                       |                      |                                         |
| ■ ■■■ main.py                  | (6 KB)               | FastAPI backend                         |
| ■■■ config/                    |                      |                                         |
| ■ ■■■ app.yaml                 | (661 B)              | Application config [stale: v1.0.0]      |
| ■■■ core/                      |                      |                                         |
| ■ ■■■ config.py                |                      | Pydantic config models                  |
| ■ ■■■ constants.py             |                      | Label constants, MODEL_ARTIFACT_FILES   |
| ■ ■■■ exceptions.py            |                      | Custom exception classes                |
| ■ ■■■ models.py                |                      | Dataclass models (AnalysisResult, etc.) |
| ■■■ data/                      |                      |                                         |
| ■ ■■■ training/                |                      | CHB-MIT EDF files (chb01..chb05)        |
| ■ ■■■ uploads/                 |                      | Temporary upload directory              |
| ■■■ eeg_processing/            |                      |                                         |
| ■ ■■■ annotations.py           | (6.5 KB)             | CHB-MIT summary parser                  |
| ■ ■■■ channel_selector.py      | (1.3 KB)             | Channel selection                       |
| ■ ■■■ feature_extraction.py    | (6.6 KB)             | 20-dim feature extraction               |
| ■ ■■■ filtering.py             | (929 B)              | FIR bandpass filter                     |
| ■ ■■■ loader.py                | (2.3 KB)             | EDF loader (MNE)                        |
| ■ ■■■ pipeline.py              | (2.1 KB)             | Shared preprocessing pipeline           |
| ■ ■■■ segmentation.py          | (2.3 KB)             | 2-second segmentation                   |
| ■ ■■■ validator.py             | (3.3 KB)             | EDF validation                          |
| ■■■ frontend/                  |                      |                                         |
| ■ ■■■ src/                     |                      |                                         |
| ■ ■ ■■■ App.tsx                | (10 KB)              | Root component + state machine          |
| ■ ■ ■■■ components/            |                      | 19 components                           |
| ■ ■ ■■■ services/api.ts        |                      | API client                              |
| ■ ■ ■■■ types/eeg.ts           |                      | TypeScript interfaces                   |
| ■ ■■■ package.json             |                      |                                         |
| ■ ■■■ vite.config.ts           |                      |                                         |
| ■■■ inference/                 |                      |                                         |
| ■ ■■■ pipeline.py              | (5.6 KB)             | InferenceService                        |
| ■■■ models/                    |                      |                                         |
| ■ ■■■ v1_0_0/ v1_1_0/ v1_2_0/  |                      |                                         |
| ■ ■■■ v1_3_0/                  | ← CURRENT PRODUCTION | (8 artifacts)                           |
| ■■■ quantum/                   |                      |                                         |
| ■ ■■■ classifier.py            | (4.1 KB)             | build_vqc, train_vqc, predict_vqc       |
| ■■■ scripts/                   |                      |                                         |
| ■■■ test_data/                 |                      | chb06_01.edf, chb07_01.edf (demo files) |
| ■■■ tests/                     |                      |                                         |
| ■ ■■■ test_api.py              |                      |                                         |
| ■ ■■■ test_checkpoint.py       |                      |                                         |
| ■■■ training/                  |                      |                                         |
| ■ ■■■ build_dataset.py         | (6.9 KB)             | Dataset construction                    |
| ■ ■■■ checkpoint.py            | (10.8 KB)            | Fold-level checkpointing                |
| ■ ■■■ evaluate.py              | (3.1 KB)             | Classification metrics                  |
| ■ ■■■ prepare_dataset.py       | (8.9 KB)             | LOSO, balancing, PCA, scaling           |
| ■ ■■■ save_artifacts.py        | (8.9 KB)             | Artifact serialization & loading        |
| ■ ■■■ train_vqc.py             | (849 B)              | Training entrypoint                     |
| ■■■ visualization/             |                      |                                         |
| ■■■ pyproject.toml             |                      |                                         |
| ■■■ diagnostic*.py             |                      | Diagnostic experiment scripts           |

## Appendix B — Technology Versions

| Technology              | Minimum Version | Source                                           |
|-------------------------|-----------------|--------------------------------------------------|
| Python                  | 3.11            | pyproject.toml: requires-python = '>=3.11'       |
| Qiskit                  | 2.0             | pyproject.toml: qiskit >= 2.0                    |
| Qiskit Machine Learning | 0.9.0           | pyproject.toml: qiskit-machine-learning >= 0.9.0 |
| MNE                     | 1.6             | pyproject.toml: mne >= 1.6                       |
| NumPy                   | 1.26            | pyproject.toml: numpy >= 1.26                    |
| SciPy                   | 1.12            | pyproject.toml: scipy >= 1.12                    |
| PyWavelets              | 1.5             | pyproject.toml: PyWavelets >= 1.5                |
| scikit-learn            | 1.4             | pyproject.toml: scikit-learn >= 1.4              |
| pandas                  | 2.1             | pyproject.toml: pandas >= 2.1                    |
| dill                    | 0.3.8           | pyproject.toml: dill >= 0.3.8                    |
| joblib                  | 1.3             | pyproject.toml: joblib >= 1.3                    |
| React                   | 18.2.0          | frontend/package.json                            |
| TypeScript              | 5.2.2           | frontend/package.json (devDependencies)          |
| Vite                    | 5.1.6           | frontend/package.json (devDependencies)          |
| Tailwind CSS            | 3.4.1           | frontend/package.json (devDependencies)          |
| Recharts                | 2.12.2          | frontend/package.json (dependencies)             |
| Lucide React            | 0.344.0         | frontend/package.json (dependencies)             |

## Appendix C — v1.3.0 Model Configuration

|                        |                                            |
|------------------------|--------------------------------------------|
| Model Name             | quantum_eeg_vqc                            |
| Version                | v1.3.0                                     |
| Created At             | 2026-08-27T12:22:37                        |
| Classifier             | VQC (Variational Quantum Classifier)       |
| Qubits                 | 4                                          |
| Feature Map            | ZZFeatureMap (reps=2, entanglement=full)   |
| Ansatz                 | RealAmplitudes (reps=3, entanglement=full) |
| Optimizer              | COBYLA (maxiter=200)                       |
| Features (pre-PCA)     | 20 (4 channels × 5 features)               |
| PCA Components         | 4                                          |
| PCA Explained Variance | 72.46% [35.68%, 20.86%, 10.91%, 5.02%]     |
| Channels               | FP1-F7, F3-C3, FZ-CZ, P4-O2                |
| Segment Duration       | 2.0 seconds                                |
| Filter                 | 0.5–40.0 Hz FIR bandpass                   |
| Decision Threshold     | 0.60                                       |
| Class Ratio (training) | 1:1 (non-seizure:seizure undersampling)    |

|                             |                                            |
|-----------------------------|--------------------------------------------|
| Training Subjects           | chb01, chb02, chb03, chb04, chb05          |
| Production Balanced Samples | 1,972                                      |
| Total Training Time         | 3,637 seconds (~60.6 minutes)              |
| Evaluation Methodology      | nested_loso_with_rotating_inner_validation |
| Total VQC Fits              | 40                                         |
| Artifact Location           | models/v1_3_0/                             |

## Appendix D — Detailed Fold Confusion Matrices

### Fold 0 — chb01

Total segments: 11,962

|                        | Predicted Non-Seizure (0) | Predicted Seizure (1) |
|------------------------|---------------------------|-----------------------|
| Actual Non-Seizure (0) | TN = 11,729               | FP = 10               |
| Actual Seizure (1)     | FN = 223                  | TP = 0                |

The model detected zero seizure segments for chb01. All 223 seizure segments were missed (FN=223). Only 10 non-seizure segments were incorrectly flagged (very few FP), resulting in 98.05% accuracy but completely failed seizure detection.

### Fold 1 — chb02

Total segments: 4,079

|                        | Predicted Non-Seizure (0) | Predicted Seizure (1) |
|------------------------|---------------------------|-----------------------|
| Actual Non-Seizure (0) | TN = 3,581                | FP = 411              |
| Actual Seizure (1)     | FN = 54                   | TP = 33               |

Some seizure detection: 33 out of 87 actual seizure segments detected (recall=37.93%). However 411 false positives give a precision of only 7.43%.

### Fold 2 — chb03

Total segments: 12,600

|                        | Predicted Non-Seizure (0) | Predicted Seizure (1) |
|------------------------|---------------------------|-----------------------|
| Actual Non-Seizure (0) | TN = 11,946               | FP = 451              |
| Actual Seizure (1)     | FN = 188                  | TP = 15               |

Only 15 seizure segments detected out of 203. High false positives (451). Recall=7.39%, Precision=3.22%.

### Fold 3 — chb04

Total segments: 19,179

|                        | Predicted Non-Seizure (0) | Predicted Seizure (1) |
|------------------------|---------------------------|-----------------------|
| Actual Non-Seizure (0) | TN = 17,572               | FP = 1,416            |
| Actual Seizure (1)     | FN = 172                  | TP = 19               |

Highest false positive count (1,416). Only 19 seizure segments detected out of 191. Very high false alarm rate from this subject.

Fold 4 — chb05

Total segments: 9,000

|                        | Predicted Non-Seizure (0) | Predicted Seizure (1) |
|------------------------|---------------------------|-----------------------|
| Actual Non-Seizure (0) | TN = 7,336                | FP = 1,382            |
| Actual Seizure (1)     | FN = 180                  | TP = 102              |

Best performing fold: 102 seizure segments detected out of 282 (recall=36.17%). Still 1,382 false positives.

References

[1] Shoeb, A.H. (2009). Application of Machine Learning to Epileptic Seizure Onset Detection and Treatment. PhD Thesis, MIT. — CHB-MIT dataset.

[2] Qiskit Contributors (2024). Qiskit: An Open-source Framework for Quantum Computing. doi:10.5281/zenodo.2573505.

[3] Qiskit Machine Learning. <https://qiskit-community.github.io/qiskit-machine-learning/>

[4] Gramfort, A. et al. (2013). MEG and EEG data analysis with MNE-Python. Frontiers in Neuroscience, 7, 267.

[5] Biamonte, J. et al. (2017). Quantum machine learning. Nature, 549, 195–202.

[6] Cerviño-Arriba, A. et al. (2021). A review of seizure detection algorithms based on EEG. Medical Engineering & Physics.

[7] Hjorth, B. (1970). EEG analysis based on time domain properties. Electroencephalography and Clinical Neurophysiology, 29(3), 306–310.

[8] PhysioNet: <https://physionet.org/content/chbmit/1.0.0/>